

DSA: Dynamic Programming - తెలుగు గైడ్ (LeetCode 150, SSE)

ఈ document చదివిన తర్వాత **DP (Dynamic Programming)** మళ్ళీ నిన్ను భయపెట్టదు. DP అంటే జనం భయపడేది – కానీ నిజం ఏమిటంటే DP ఒక *technique* కాదు, ఒక **ఆలోచనా విధానం (way of thinking)**. కింద మనం మొదట ఆ ఆలోచనా విధానాన్ని (universal 5-step recipe) లోతుగా నేర్చుకుంటాం, తర్వాత దాన్ని 14 classic LeetCode problems మీద పదే పదే apply చేసి – నీ brain లో ఆ pattern permanent గా ముద్రించుకునేలా చేస్తాం. ప్రతి problem కి: **ఎలా ఆలోచించాలి** (state, recurrence, base case explicit గా), brute-force recursion ఎందుకు slow, memoization → tabulation, clean JavaScript, dry run (dp table నింపడం), complexity, గుర్తుంచుకోవాల్సినది, సాధారణ తప్పులు. లక్ష్యం: **"ఒకసారి చదివితే మర్చిపోకూడదు."**

ముఖ్య గమనిక: నీకు DSA అన్నలు తెలియకపోతే, ముందు [DSA_00_Foundations_Telugu.md](#) చదువు – అందులో recursion అంటే ఏమిటి, call stack, Big-O, time/space complexity, arrays/strings basics ఉన్నాయి. DP recursion మీద కట్టబడింది, కాబట్టి recursion comfortable అయ్యాకే ఇది చదవడం మంచిది. ఇక్కడ మనం LeetCode Top-Interview-150 లోని **Dynamic Programming (1D + Multidimensional)** section (14 problems) ని senior (SSE) interview స్థాయిలో పట్టేస్తాం. Solutions అన్నీ **JavaScript (ES2020+)** లో, runnable & tested.

విషయ సూచిక (Table of Contents)

మొదట DP ఆలోచనా విధానం నేర్చుకో (ఇదే అన్నిటికీ పునాది):

- Pattern: Dynamic Programming – ఎలా ఆలోచించాలి (universal 5-step recipe, memoization vs tabulation, space optimization)
- Pattern: 2D DP (grid dp + string dp – $dp[i][j]$ meaning, filling order)

1D DP Problems:

1. Climbing Stairs (#70) – Easy
2. House Robber (#198) – Medium
3. Word Break (#139) – Medium
4. Coin Change (#322) – Medium
5. Longest Increasing Subsequence (#300) – Medium

Multidimensional (2D) DP Problems:

6. Triangle (#120) – Medium
7. Minimum Path Sum (#64) – Medium
8. Unique Paths II (#63) – Medium
9. Longest Palindromic Substring (#5) – Medium
10. Interleaving String (#97) – Medium

11. Edit Distance (#72) – Hard
12. Maximal Square (#221) – Medium
13. Best Time to Buy and Sell Stock III (#123) – Hard
14. Best Time to Buy and Sell Stock IV (#188) – Hard

Pattern: Dynamic Programming – ఎలా ఆలోచించాలి

ఇదే ఈ document లో అత్యంత ముఖ్యమైన section. ఇక్కడ ఏ ఒక్క problem గురించీ కాదు – ఏ DP problem నైనా ఎలా విడగొట్టాలో నేర్చుకుంటాం. ఇది ఒంటబట్టితే, కింది 14 problems కేవలం "అదే recipe ని మళ్ళీ మళ్ళీ apply చెయ్యడం" అవుతుంది.

వివరణ – DP అంటే నిజంగా ఏమిటి?

DP ని ఒక్క వాక్యంలో చెప్పాలంటే: "ఒక subproblem ని మళ్ళీ మళ్ళీ solve చెయ్యకుండా, ఒకసారి solve చేసి, answer ని గుర్తుపెట్టుకుని (remember), తర్వాత reuse చెయ్యడం." అంతే. "Dynamic Programming" అనే భయపెట్టే పేరు వెనుక ఉన్న idea ఇంత simple.

DP రెండు లక్షణాలు ఉన్న problems మీద పని చేస్తుంది:

1. **Overlapping subproblems** – పెద్ద problem ని చిన్నవిగా విడగొడితే, అవే చిన్న subproblems పదే పదే వస్తాయి. (ఉదా: Fibonacci లో $\text{fib}(3)$ ని లెక్కించడానికి $\text{fib}(2)$, $\text{fib}(1)$; $\text{fib}(4)$ కి కూడా $\text{fib}(2)$ మళ్ళీ కావాలి – same subproblem పునరావృతం.)
2. **Optimal substructure** – పెద్ద problem యొక్క optimal answer, దాని subproblems యొక్క optimal answers నుండి కట్టవచ్చు. (ఉదా: n-వ మెట్టుకి చేరే మార్గాల సంఖ్య = (n-1)-వ మెట్టు మార్గాలు + (n-2)-వ మెట్టు మార్గాలు.)

Real-life analogy: నువ్వు ఒక పెద్ద పరీక్షకి చదువుతున్నావు. ప్రతి chapter లో కొన్ని sums వస్తాయి. కొన్ని sub-calculations (ఉదా: "log 2 విలువ") పదే పదే అవసరం అవుతాయి. తెలివైన student ప్రతిసారీ మళ్ళీ లెక్కించడు – ఒకసారి లెక్కించి, ఒక **cheat-sheet (memo)** లో రాసుకుని, మళ్ళీ అవసరం అయినప్పుడు అక్కడ చూసుకుంటాడు. DP అంటే అదే: **once compute, forever reuse**. ఆ cheat-sheet నే మనం dp array/table అంటాం.

ఇది DP problem అని ఎలా గుర్తించాలి? (recognition signals)

కింది సంకేతాలు కనిపిస్తే, "ఇది DP కావచ్చు" అని అనుమానించు:

- **ప్రతి step దగ్గర choices (options)** ఉంటాయి – "ఈ coin తీసుకోవాలా వద్దా", "ఈ ఇంట్లో దొంగతనం చెయ్యాలా వద్దా", "1 మెట్టు ఎక్కాలా 2 మెట్లా".
- ప్రశ్న ఇలా ఉంటుంది: **maximum / minimum** (గరిష్ట profit, కనిష్ట coins/cost/path), **count** (ఎన్ని ways), లేదా **true/false** (ఇది సాధ్యమా – can we form / can we reach / can we partition).
- **"ప్రతి subset / subsequence / partition / combination"** అని brute-force చేస్తే exponential (2^n , $n!$) అవుతోంది – కానీ choices మధ్య structure (order/monotonicity) ఉంది.

- Greedy (ప్రతి step best local choice) **తప్పు answer** ఇస్తోంది — ఎందుకంటే ఇప్పటి choice భవిష్యత్తుని ప్రభావితం చేస్తోంది. అప్పుడు అన్ని choices ని explore చేసి, overlapping ని memoize చెయ్యాలి = DP.

Greedy vs DP గుర్తుంచుకో: Greedy = "ఇప్పుడు best అనిపించింది తీసుకో, వెనక్కి చూడకు" (Coin Change లో ఇది తప్పు: $[1, 3, 4]$, amount 6 $\rightarrow$ greedy $4+1+1=3$ coins, కానీ optimal $3+3=2$ coins). DP = "అన్ని choices ప్రయత్నించి, best ని remember చెయ్యి."

The 5-Step DP Recipe (ఇదే మంత్రం — ప్రతి problem కి ఇవే 5 అడుగులు)

ఏ DP problem వచ్చినా, ఈ 5 ప్రశ్నలకి క్రమంగా జవాబు ఇస్తే — solution దానంతట అదే బయటకు వస్తుంది:

అడుగు	ప్రశ్న	ఏం చెయ్యాలి
1. State	dp[i] అంటే సరిగ్గా ఏమిటి?	ఒక్క index/parameter తో ఒక subproblem ని define చెయ్యి. ఇదే అత్యంత కీలకమైన అడుగు. State తప్పితే మిగతా అంతా తప్పు.
2. Recurrence	పెద్ద dp[i] ని చిన్న dp[<i] నుండి ఎలా కడతాం?	ఈ state దగ్గర ఉన్న choices అన్నీ రాసి, వాటిలో max/min/sum/or తీసుకో.
3. Base case(s)	అన్నిటికన్నా చిన్న subproblem యొక్క answer నేరుగా ఏమిటి?	Recursion ఆగే చోటు. (ఉదా: dp[0], ఖాళీ string, amount 0.)
4. Order	dp values ని ఏ క్రమంలో నింపాలి?	ప్రతి dp[i] ని లెక్కించే ముందు, అది ఆధారపడే చిన్న subproblems ఇప్పటికే ready గా ఉండాలి (small $\rightarrow$ large).
5. Answer	Final జవాబు table లో ఎక్కడ ఉంది?	సాధారణంగా dp[n] లేదా dp[m][n], కొన్నిసార్లు $\max(dp[\dots])$.

ఈ recipe ని Fibonacci $\rightarrow$ Climbing Stairs మీద చూద్దాం

Fibonacci తో మొదలుపెడదాం (DP యొక్క "hello world"). $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$, $\text{fib}(0)=0$, $\text{fib}(1)=1$.

Naive recursion ఇలా ఉంటుంది:

```
function fib(n) {
  if (n <= 1) return n;           // base case
  return fib(n - 1) + fib(n - 2); // recurrence
}
```

ఇది **exponential $O(2^n)$** — ఎందుకు? $\text{fib}(5)$ ని draw చేస్తే, $\text{fib}(3)$ రెండుసార్లు, $\text{fib}(2)$ మూడుసార్లు లెక్కించబడుతుంది. ఇదే **overlapping subproblems**. అదే వృథా ని DP తొలగిస్తుంది.

ఇప్పుడు 5-step recipe apply చేద్దాం:

- State:** $\text{dp}[i]$ = i-వ Fibonacci number.
- Recurrence:** $\text{dp}[i] = \text{dp}[i-1] + \text{dp}[i-2]$.

3. **Base:** $dp[0] = 0$, $dp[1] = 1$.
4. **Order:** $i = 2$ నుండి n వరకు (ఎడమ నుండి కుడికి – $dp[i-1]$, $dp[i-2]$ ముందే ready).
5. **Answer:** $dp[n]$.

Climbing Stairs (Problem 1) exact గా same structure – కేవలం base cases మారతాయి ($dp[0]=1$, $dp[1]=1$, ఎందుకంటే "ways to reach"). అందుకే మనం మొదటి problem గా దాన్ని పెట్టాం.

రెండు రూపాలు: Memoization (Top-Down) vs Tabulation (Bottom-Up)

ఒకే recurrence ని రెండు రకాలుగా code చెయ్యవచ్చు – రెండూ same complexity, రెండూ correct. ఏది వాడాలో నీ ఇష్టం (interview లో ఏదైనా OK).

(A) Memoization = Recursion + cache (Top-Down). Naive recursion ని అలాగే ఉంచి, ఒక memo cache పెడతాం. Subproblem ని ఒకసారి solve చేశాక cache లో పెట్టి, మళ్ళీ అడిగితే cache నుండి ఇస్తాం. "పైనుండి కిందకి" – పెద్ద problem నుండి మొదలై చిన్నవాటిని అడుగుతుంది.

```
// Top-Down template (memoization)
function solve(i, memo = new Map()) {
  if (/* base case */) return /* base value */;
  if (memo.has(i)) return memo.get(i); // ← ఇప్పటికే లెక్కించామా? cache నుండి ఇప్పు
  let ans = /* choices ని combine: solve(smaller) లను వాడి recurrence */;
  memo.set(i, ans); // ← లెక్కించాక cache లో పెట్టు
  return ans;
}
```

(B) Tabulation = Loop + array (Bottom-Up). Recursion అస్సలు లేకుండా, ఒక dp array తీసుకుని, base cases నింపి, చిన్న → పెద్ద క్రమంలో loop తో నింపుతాం. "కిందనుండి పైకి" – చిన్న subproblems ముందు, పెద్దవి తర్వాత.

```
// Bottom-Up template (tabulation)
const dp = new Array(n + 1);
dp[0] = /* base */; dp[1] = /* base */; // base case(s)
for (let i = 2; i <= n; i++) {
  dp[i] = /* recurrence: dp[i-1], dp[i-2] ... నుండి */;
}
return dp[n]; // answer
```

అంశం	Memoization (Top-Down)	Tabulation (Bottom-Up)
రూపం	Recursion + cache	Loop + array
రాయడం సులభమా	State/recurrence స్పష్టంగా ఉంటే సులభం	Order జాగ్రత్తగా ఆలోచించాలి
Overhead	Recursion call stack (deep అయితే stack overflow risk)	Loop మాత్రమే – fast, stack safe
అన్ని states compute అవుతాయా	అవసరమైనవి మాత్రమే (lazy)	అన్నీ compute (కొన్ని అనవసరం కావచ్చు)
Space optimize	కష్టం	సులభం (rolling variables)

Space Optimization (చాలా problems లో $O(n) \rightarrow O(1)$ లేదా $O(n^2) \rightarrow O(n)$)

Tabulation లో $dp[i]$ కేవలం చివరి కొన్ని values మీద (ఉదా: $dp[i-1]$, $dp[i-2]$) ఆధారపడితే, పూర్తి array అవసరం లేదు – ఆ రెండు మూడు values ని variables లో ఉంచితే చాలు. Fibonacci ని $O(n)$ space నుండి $O(1)$ కి తగ్గించడం:

```
function fib(n) {
  if (n <= 1) return n;
  let twoBack = 0, oneBack = 1; // dp[i-2], dp[i-1]
  for (let i = 2; i <= n; i++) {
    const cur = oneBack + twoBack; // dp[i]
    twoBack = oneBack; // shift: పాత dp[i-1] ఇప్పుడు dp[i-2] అవుతుంది
    oneBack = cur; // పాత dp[i] ఇప్పుడు dp[i-1]
  }
  return oneBack;
}
```

2D DP లో ఇదే idea: $dp[i][j]$ కేవలం previous row ($dp[i-1]$) మీద ఆధారపడితే, పూర్తి 2D matrix కాకుండా ఒక 1D row ఉంచితే చాలు $\rightarrow O(m \cdot n)$ space నుండి $O(n)$.

Complexity – DP ఎంత fast?

సూత్రం: **DP time = (states సంఖ్య) $\times$ (ఒక్క state ని లెక్కించే work).**

- Fibonacci/Climbing Stairs: n states $\times O(1)$ work = **$O(n)$** .
- Coin Change: $(amount+1)$ states $\times O(coins)$ work = **$O(amount \times coins)$** .
- Edit Distance: $(m+1)(n+1)$ states $\times O(1)$ = **$O(m \cdot n)$** .

ఈ ఒక్క సూత్రం interview లో "complexity ఎంత?" అని అడిగినప్పుడు వెంటనే జవాబు ఇస్తుంది – states లెక్కపెట్టు, ఒక్క state work తో గుణించు.

గుర్తుంచుకో (ఈ primer సారాంశం): ఏ DP అయినా = **State ని define చెయ్యి** $\rightarrow$ **Recurrence రాయ్** $\rightarrow$ **Base case పెట్టు** $\rightarrow$ **Order నిర్ణయించు** $\rightarrow$ **Answer తీసుకో**. State definition crystal-clear అయితే, మిగతా 90% పని అయిపోయినట్టే. కింది ప్రతి problem లో మనం ఈ 5 అడుగులు explicit గా రాస్తాం – నువ్వు కూడా interview లో ఇలాగే బిగ్గరగా (out loud) ఆలోచించు.

1. Climbing Stairs (LeetCode #70) – Easy

- సమస్య:** నువ్వు ఒక మెట్లు (staircase) కింద ఉన్నావు, పైకి n మెట్లు ఉన్నాయి. ప్రతిసారీ నువ్వు **1 మెట్లు లేదా 2 మెట్లు** ఎక్కగలవు. పైకి $(n-వ మెట్టుకి)$ చేరడానికి **ఎన్ని distinct మార్గాలు (ways)** ఉన్నాయి?
- Key constraints:** $1 \leq n \leq 45$. Answer ఎప్పుడూ positive integer.
- ఉదాహరణ:**

Input: $n = 3$

Output: 3

వివరణ: $\{1+1+1\}$, $\{1+2\}$, $\{2+1\}$ – మూడు మార్గాలు.

- **ఎలా ఆలోచించాలి (5-step recipe):**

అన్నిటికన్నా ముఖ్యమైన insight: **n -వ మెట్టుకి చేరాలంటే, నా చివరి అడుగు ఏది?** రెండే అవకాశాలు – నేను $(n-1)$ -వ మెట్టు నుండి 1 అడుగు వేశాను, **లేదా** $(n-2)$ -వ మెట్టు నుండి 2 అడుగులు వేశాను. కాబట్టి n -వ మెట్టు మార్గాలు = $(n-1)$ మార్గాలు + $(n-2)$ మార్గాలు. ఇదే Fibonacci!

1. **State:** $dp[i]$ = i -వ మెట్టుకి చేరే distinct మార్గాల సంఖ్య.
2. **Recurrence:** $dp[i] = dp[i-1] + dp[i-2]$ (చివరి అడుగు 1 అయితే $dp[i-1]$ నుండి, 2 అయితే $dp[i-2]$ నుండి – రెండూ కలిపి).
3. **Base case:** $dp[0] = 1$ (ground మీద ఉండటం ఒక్కటే way – ఏమీ చెయ్యకుండా), $dp[1] = 1$ (ఒక్క మెట్టు: $\{1\}$ ఒక్కటే).
4. **Order:** $i = 2$ నుండి n వరకు ($dp[i-1]$, $dp[i-2]$ ముందే ready).
5. **Answer:** $dp[n]$.

- **Brute Force (recursion):** పైన చెప్పిన recurrence ని నేరుగా recursion గా రాస్తే:

```
function climb(n) {  
  if (n <= 1) return 1;           // base: dp[0]=1, dp[1]=1  
  return climb(n - 1) + climb(n - 2);  
}
```

ఇది $O(2^n)$ – ఎందుకు slow? $climb(5)$ కి $climb(3)$ రెండుసార్లు, $climb(2)$ మూడుసార్లు లెక్కించబడుతుంది. అవే **overlapping subproblems**. $n=45$ కి ఇది కోట్ల calls – TLE.

- **Optimal Approach:**

- **Memoization (top-down):** పై recursion కే ఒక memo array జోడిస్తే, ప్రతి $climb(k)$ ఒక్కసారే లెక్కించబడి cache అవుతుంది $\rightarrow O(n)$ time.
- **Tabulation (bottom-up):** dp array ని base నుండి loop తో నింపు $\rightarrow O(n)$ time, $O(n)$ space.
- **Space-optimized:** $dp[i]$ కి కేవలం చివరి రెండు values చాలు $\rightarrow O(1)$ space (కింది code).

- **Solution (JavaScript):**

```
/**  
 * @param {number} n - మెట్ల సంఖ్య  
 * @return {number} - distinct మార్గాల సంఖ్య  
 */  
* Tabulation + space optimization:  $dp[i] = dp[i-1] + dp[i-2]$ .  
* పూర్తి array బదులు రెండు variables చాలు.  
*/  
var climbStairs = function (n) {  
  // twoBack =  $dp[i-2]$ , oneBack =  $dp[i-1]$ . మొదట  $dp[0]=1$ ,  $dp[1]=1$ .  
  let twoBack = 1;  
  let oneBack = 1;
```

```

for (let i = 2; i <= n; i++) {
  const cur = oneBack + twoBack; // dp[i] = dp[i-1] + dp[i-2]
  twoBack = oneBack;           // shift ముందుకు
  oneBack = cur;
}

return oneBack; // n=1 అయితే loop నడవదు → oneBack = 1 (సరైనది)
};

/* Memoization version (top-down) – same complexity, reference కోసం:
var climbStairs = function (n, memo = [1, 1]) {
  if (memo[n] !== undefined) return memo[n];
  memo[n] = climbStairs(n - 1, memo) + climbStairs(n - 2, memo);
  return memo[n];
}; */

```

- **Dry Run:** $n = 5$.

i	$dp[i] = dp[i-1] + dp[i-2]$	value	(twoBack, oneBack) after
0	base	1	—
1	base	1	(1, 1)
2	$1 + 1$	2	(1, 2)
3	$2 + 1$	3	(2, 3)
4	$3 + 2$	5	(3, 5)
5	$5 + 3$	8	(5, 8)

Return 8 .  (n=5 కి 8 మార్గాలు.)

- **Complexity:** Time $O(n)$ — ప్రతి మెట్టు ఒక్కసారే process. Space $O(1)$ — రెండు variables మాత్రమే (tabulation array అయితే $O(n)$).
- **గుర్తుంచుకోవాల్సినది:** $dp[i] = dp[i-1] + dp[i-2]$ — ఇది **Fibonacci in disguise**. "చివరి అడుగు ఏది?" అని అడిగితే choices బయటపడతాయి. "ఎన్ని ways" + "ప్రతి step 1 లేదా 2 choices" = ఈ pattern. House Robber, Decode Ways కూడా ఇదే కుటుంబం.
- **సాధారణ తప్పులు:**
 - **Base case తప్పు:** $dp[0] = 0$ పెట్టడం. "0 మెట్లకి 0 ways" అనిపిస్తుంది కానీ recurrence పని చెయ్యాలంటే $dp[0] = 1$ కావాలి (ఏమీ చెయ్యని ఒక్క empty way). ఇది off-by-base classic తప్పు.
 - **n=1 edge case:** loop $i=2$ నుండి మొదలైతే $n=1$ కి నడవదు — అప్పుడు initial $oneBack=1$ నే return కావాలి. దీన్ని miss చేస్తే crash లేదా తప్పు.
 - Memoization లో $memo[n] !== undefined$ కి బదులు $if (memo[n])$ వాడితే value 0 ఉన్నప్పుడు bug (ఇక్కడ 0 రాదు కానీ అలవాటుగా జాగ్రత్త).

2. House Robber (LeetCode #198) — Medium

- **సమస్య:** ఒక వీధిలో వరుసగా ఇళ్ళు ఉన్నాయి, $\text{nums}[i] = i$ -వ ఇంట్లో ఉన్న డబ్బు. నువ్వు దొంగవి — కానీ **రెండు పక్కపక్క (adjacent) ఇళ్ళు దోచుకుంటే** alarm మోగుతుంది (police వస్తారు). Alarm మోగకుండా దోచుకోగల గరిష్ట డబ్బు ఎంత?
- **Key constraints:** $1 \leq \text{nums.length} \leq 100$, $0 \leq \text{nums}[i] \leq 400$.
- **ఉదాహరణ:**

```
Input:  nums = [2,7,9,3,1]
Output: 12
వివరణ: ఇల్లు 0 (2) + ఇల్లు 2 (9) + ఇల్లు 4 (1) = 12. (adjacent కాదు)
```

- **ఎలా ఆలోచించాలి (5-step recipe):**

Climbing Stairs లాగే, కీలక ప్రశ్న: **చివరి ఇల్లు (i) దగ్గర నా choice ఏమిటి?** రెండే options —

- **ఈ ఇల్లు దోచుకో:** అప్పుడు (i-1) దోచుకోకూడదు (adjacent). కాబట్టి డబ్బు = $\text{nums}[i] + \text{dp}[i-2]$.
 - **ఈ ఇల్లు వదిలేయ్:** అప్పుడు $\text{answer} = \text{dp}[i-1]$ (అంతకుముందు దాకా best). రెండింటిలో గరిష్టం తీసుకో. ఇక్కడ greedy ("ప్రతి రిచ్ ఇల్లు దోచుకో") పని చెయ్యదు — ఎందుకంటే ఇప్పటి choice తర్వాతి choice ని block చేస్తుంది. అందుకే DP.
1. **State:** $\text{dp}[i]$ = ఇల్లు 0 నుండి i వరకు మాత్రమే చూసినప్పుడు (adjacent constraint తో) దోచుకోగల గరిష్ట డబ్బు.
 2. **Recurrence:** $\text{dp}[i] = \max(\text{dp}[i-1], \text{nums}[i] + \text{dp}[i-2])$ — (వదిలేయ్) vs (దోచుకో).
 3. **Base case:** $\text{dp}[0] = \text{nums}[0]$ (ఒక్క ఇల్లు → అదే దోచుకో), $\text{dp}[1] = \max(\text{nums}[0], \text{nums}[1])$ (రెండిట్లో పెద్దది).
 4. **Order:** $i = 2$ నుండి $n-1$ వరకు.
 5. **Answer:** $\text{dp}[n-1]$ (చివరి ఇల్లు దాకా best).
- **Brute Force (recursion):** ప్రతి ఇంటికి "దోచుకో / వదిలేయ్" — 2^n subsets:

```
function rob(nums, i = nums.length - 1) {
  if (i < 0) return 0; // base: ఇల్లు లేదు
  return Math.max(
    rob(nums, i - 1), // ఈ ఇల్లు వదిలేయ్
    nums[i] + rob(nums, i - 2) // ఈ ఇల్లు దోచుకో → i-1 skip
  );
}
```

$O(2^n)$ — $\text{rob}(i-1)$ మరియు $\text{rob}(i-2)$ రెండూ లోపల మళ్ళీ చిన్న indices ని పదే పదే అడుగుతాయి = overlapping subproblems.

- **Optimal Approach:**
 - **Memoization:** పై recursion + $\text{memo}[i]$ cache → $O(n)$.
 - **Tabulation:** dp array ని base నుండి నింపు → $O(n)$ time, $O(n)$ space.

- **Space-optimized:** $dp[i]$ కి $dp[i-1]$, $dp[i-2]$ చాలు $\rightarrow$ **$O(1)$ space** (కొది code). ఇది Fibonacci-style rolling.

- **Solution (JavaScript):**

```
/**
 * @param {number[]} nums - ఇళ్ళలో డబ్బు
 * @return {number} - దోచుకోగల గరిష్ట డబ్బు (adjacent కాకుండా)
 */
*
* dp[i] = max(dp[i-1], nums[i] + dp[i-2]).
* prev1 = dp[i-1], prev2 = dp[i-2]. మొదట రెండూ 0.
*/
var rob = function (nums) {
  let prev2 = 0; // dp[i-2]: రెండు ఇళ్ళ కిందటి best
  let prev1 = 0; // dp[i-1]: ఒక ఇంటి కిందటి best

  for (const money of nums) {
    // ఈ ఇంటిని దోచుకుంటే prev2 + money, లేకపోతే prev1 (కిందటిది).
    const cur = Math.max(prev1, prev2 + money);
    prev2 = prev1; // shift ముందుకు
    prev1 = cur;
  }

  return prev1; // అన్ని ఇళ్ళూ చూశాక best
};
```

గమనిక: $prev1$, $prev2$ రెండూ 0 తో మొదలుపెట్టడం వల్ల empty array, single house — అన్ని edge cases automatic గా handle అవుతాయి. ప్రత్యేక if conditions అవసరం లేదు. ఇది elegant.

- **Dry Run:** $nums = [2, 7, 9, 3, 1]$.

money	$cur = \max(prev1, prev2 + money)$	$(prev2, prev1)$ after
2	$\max(0, 0+2) = 2$	(0, 2)
7	$\max(2, 0+7) = 7$	(2, 7)
9	$\max(7, 2+9) = 11$	(7, 11)
3	$\max(11, 7+3) = 11$	(11, 11)
1	$\max(11, 11+1) = 12$	(11, 12)

Return 12 . 

- **Complexity:** Time $O(n)$ — ఒక్క pass. Space $O(1)$ — రెండు variables.
- **గుర్తుంచుకోవాల్సినది:** $dp[i] = \max(\text{take} = nums[i] + dp[i-2], \text{skip} = dp[i-1])$. "Adjacent తీసుకోకూడదు / consecutive select చెయ్యకూడదు" అనే ఏ problem కైనా ఈ take-or-skip pattern. (House Robber II circular version, Delete and Earn ఇదే base.)
- **సాధారణ తప్పులు:**

- `dp[i-2]` బదులు `dp[i-1]` వాడటం "take" లో – అప్పుడు adjacent constraint violate. Take చేసినప్పుడు తప్పకుండా రెండు వెనక్కి (prev2) వెళ్ళాలి.
- **Base case** `dp[1] = nums[1]` అని రాయడం – తప్పు! `dp[1] = max(nums[0], nums[1])` కావాలి (మొదటి రెండిట్లో పెద్దది). Rolling-variable version ఇది automatic గా handle చేస్తుంది కాబట్టి safer.
- Empty array కి `nums[0]` access చేసి crash – rolling version లో ఈ problem లేదు (loop నడవదు, 0 return).

3. Word Break (LeetCode #139) – Medium

- **సమస్య:** ఒక string `s` మరియు ఒక dictionary `wordDict` (words list) ఇచ్చారు. `s` ని dictionary లోని words యొక్క **వరుస (sequence)** గా విడగొట్టవచ్చా? (ఒక్కో word ని ఎన్నిసార్లైనా వాడొచ్చు, అన్ని అక్షరాలూ వాడాలి.) `true` / `false` return చెయ్యి.
- **Key constraints:** `1 <= s.length <= 300`, dictionary words distinct. అక్షరాలు lowercase.
- **ఉదాహరణ:**

Input: `s = "leetcode", wordDict = ["leet","code"]`
 Output: `true` ("leet" + "code")

Input: `s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]`
 Output: `false` (ఎలా విడగొట్టినా "og" లేదా leftover మిగులుతుంది)

- **ఎలా ఆలోచించాలి (5-step recipe):**

ఇది **true/false DP** (సాధ్యమా?). కీలక ప్రశ్న: **string మొదటి `i` అక్షరాలు (`s[0..i-1]`) విడగొట్టవచ్చా?** దీన్ని చిన్న subproblem తో connect చెయ్యాలంటే – ఒక **cut point** `j` ఊహించు: `s[0..j-1]` విడగొట్టబడితే (`dp[j] = true`), మరియు మిగిలిన భాగం `s[j..i-1]` ఒక dictionary word అయితే → `s[0..i-1]` కూడా విడగొట్టవచ్చు. ఏ ఒక్క `j` పని చేసినా చాలు (OR).

1. **State:** `dp[i] = true` అయితే string యొక్క మొదటి `i` అక్షరాలు (`s.substring(0, i)`) dictionary words గా విడగొట్టవచ్చు.
 2. **Recurrence:** `dp[i] = OR over j in [0, i) of (dp[j] AND s.substring(j, i) ∈ dict)`. అంటే ఏదో ఒక cut `j` ఉండాలి, ఎక్కడ ఎడమ భాగం విడగొట్టబడి, కుడి భాగం ఒక word అయ్యేలా.
 3. **Base case:** `dp[0] = true` – ఖాళీ string ఎప్పుడూ (trivially) విడగొట్టబడుతుంది.
 4. **Order:** `i = 1` నుండి `n` వరకు; ప్రతి `i` కి `j = 0` నుండి `i-1` (అన్ని cut points).
 5. **Answer:** `dp[n]` (మొత్తం string).
- **Brute Force (recursion):** ప్రతి prefix ని word గా try చేసి, మిగిలిన suffix ని recursively:

```
function wordBreak(s, words) {
  const set = new Set(words);
  function can(start) {
    if (start === s.length) return true; // అన్ని అక్షరాలూ వాడేశాం
    for (let end = start + 1; end <= s.length; end++) {
```

```

    if (set.has(s.substring(start, end)) && can(end)) return true;
  }
  return false;
}
return can(0);
}

```

Worst case exponential — same `start` index పదే పదే different పైనుండి రావడం = overlapping subproblems.
(ఉదా: "aaaa...a" + words ["a", "aa"] .)

- **Optimal Approach:**

- **Memoization:** పై `can(start)` కి `memo[start]` cache జోడిస్తే $\rightarrow O(n^2)$ ($\times$ substring cost).
- **Tabulation:** `dp[i]` array, రెండు nested loops $(i, j) \rightarrow O(n^2)$ subproblems, ఒక్కో substring create/lookup $O(n)/O(1)$. మొత్తం సుమారు $O(n^2)$ (word length bound ఉంటే $O(n \cdot m)$).

- **Solution (JavaScript):**

```

/**
 * @param {string} s
 * @param {string[]} wordDict
 * @return {boolean}
 */
*
* dp[i] = s యొక్క మొదటి i అక్షరాలు విడగొట్టవచ్చా.
* dp[i] = OR_j (dp[j] && s[j..i] ఒక word).
*/
var wordBreak = function (s, wordDict) {
  const words = new Set(wordDict); // O(1) lookup కోసం Set
  const n = s.length;
  const dp = new Array(n + 1).fill(false);
  dp[0] = true; // ఖాళీ string  $\rightarrow$  true (base)

  for (let i = 1; i <= n; i++) {
    for (let j = 0; j < i; j++) {
      // ఎడమ భాగం [0..j) విడగొట్టబడిందా, కుడి భాగం [j..i] ఒక word అయిందా?
      if (dp[j] && words.has(s.substring(j, i))) {
        dp[i] = true;
        break; // ఒక్క valid cut చాలు - మిగతా j లు చూడక్కర్లేదు
      }
    }
  }

  return dp[n];
};

```

- **Dry Run:** `s = "leetcode"` , `dict = {leet, code}` . (i = index into `s`, 1-based length)

i	prefix s[0..i)	ఏ j పని చేసింది?	dp[i]
0	""	base	true
4	"leet"	j=0: dp[0]=true & "leet" ∈ dict	true
8	"leetcode"	j=4: dp[4]=true & "code" ∈ dict	true

(i = 1,2,3,5,6,7 అన్నీ false — ఏ valid cut లేదు.) Return dp[8] = true . ✓

- **Complexity:** Time $O(n^2)$ subproblem pairs (× substring $O(n)$ worst = $O(n^3)$), కానీ dictionary word max length L తో bound చేస్తే $O(n \cdot L)$. Space $O(n)$ dp + dict Set.
- **గుర్తుంచుకోవాల్సినది:** $dp[i] = \exists j : dp[j] \ \&\& \ s[j..i] \in \text{dict}$. "String ని valid pieces గా విడగొట్టవచ్చా / segment చెయ్యవచ్చా" అనే ఏ problem కైనా ఈ prefix-DP + cut-point pattern. (Word Break II, Palindrome Partitioning కూడా ఇదే base.)
- **సాధారణ తప్పులు:**
 - $dp[0] = \text{false}$ పెట్టడం — అప్పుడు ఏదీ true కాదు. ఖాళీ string base తప్పకుండా **true** .
 - **Index confusion:** $dp[i]$ అంటే "index i" కాదు, "మొదటి i అక్షరాలు (length i)". $s.\text{substring}(j, i)$ — j నుండి i-1 వరకు అక్షరాలు. ఈ 1-offset ని జాగ్రత్తగా చూడు (off-by-one classic).
 - Dictionary ని **Array** గా ఉంచి **.includes()** వాడటం — $O(n)$ lookup, మొత్తం slow. తప్పకుండా **Set** .

4. Coin Change (LeetCode #322) — Medium

- **సమస్య:** వేర్వేరు విలువల coins coins[] (unlimited supply, ఒక్కో coin ఎన్నిసార్లైనా వాడొచ్చు) మరియు ఒక target amount ఇచ్చారు. ఆ amount ని కూడబెట్టడానికి కావాల్సిన **కనిష్ట coins సంఖ్య** ఎంత? సాధ్యం కాకపోతే **-1** return.
- **Key constraints:** $1 \leq \text{coins.length} \leq 12$, $0 \leq \text{amount} \leq 10^4$.
- **ఉదాహరణ:**

```

Input: coins = [1,2,5], amount = 11
Output: 3      (5 + 5 + 1 = 11, మూడు coins)

Input: coins = [2], amount = 3
Output: -1     (2 తో 3 కూడబెట్టలేం)

```

- **ఎలా ఆలోచించాలి (5-step recipe):**

ఇది **minimize DP** (కనిష్ట). ఇక్కడ **greedy తప్పు** అని మొదట అర్థం చేసుకో: "ఎప్పుడూ పెద్ద coin తీసుకో" అంటే coins = [1,3,4], amount=6 → greedy 4+1+1 = 3 coins , కానీ optimal 3+3 = 2 coins . కాబట్టి అన్ని choices try చెయ్యాలి = DP.

కీలక ప్రశ్న: **amount a ని కూడబెట్టడానికి, నా చివరి coin ఏది?** అది coins లో ఏదైనా c కావచ్చు. ఆ చివరి coin c అయితే, మిగిలిన $a - c$ ని కూడబెట్టడానికి కనిష్ట coins = $dp[a-c]$, దానికి ఈ ఒక్క coin కలిపి $dp[a-c] + 1$. అన్ని

coins మీద కనిష్టం తీసుకో.

1. **State:** $dp[a]$ = amount a ని కూడబెట్టడానికి కావాల్సిన కనిష్ట coins సంఖ్య.
 2. **Recurrence:** $dp[a] = \min \text{ over } c \text{ in coins of } dp[a - c] + 1$ (where $c \leq a$).
 3. **Base case:** $dp[0] = 0$ — 0 amount కి 0 coins చాలు.
 4. **Order:** $a = 1$ నుండి amount వరకు ($dp[a-c]$ ఎప్పుడూ చిన్నది, ముందే ready).
 5. **Answer:** $dp[\text{amount}]$; ఇది **Infinity** గా మిగిలితే (ఏ combination పని చెయ్యలేదు) → **-1**.
- **Brute Force (recursion):** ప్రతి coin ని చివరిగా try చేసి, మిగిలిన amount ని recursively:

```
function coinChange(coins, amount) {
  function minCoins(rem) {
    if (rem === 0) return 0;           // base
    if (rem < 0) return Infinity;      // invalid path
    let best = Infinity;
    for (const c of coins) best = Math.min(best, minCoins(rem - c) + 1);
    return best;
  }
  const ans = minCoins(amount);
  return ans === Infinity ? -1 : ans;
}
```

Exponential — ఒకే rem value వేర్వేరు coin sequences ద్వారా పదే పదే లెక్కించబడుతుంది = overlapping.

- **Optimal Approach:**

- **Memoization:** $minCoins(rem)$ కి $memo[rem] \rightarrow O(\text{amount} \times \text{coins})$.
- **Tabulation:** $dp[0..\text{amount}]$ array, ప్రతి amount కి అన్ని coins try → **$O(\text{amount} \times \text{coins})$** time, $O(\text{amount})$ space. ఇది "unbounded knapsack" family.

- **Solution (JavaScript):**

```
/**
 * @param {number[]} coins
 * @param {number} amount
 * @return {number} - కనిష్ట coins, లేకపోతే -1
 */
*  $dp[a]$  = amount  $a$  కి కనిష్ట coins.
*  $dp[a] = \min_c (dp[a-c] + 1)$ .  $Infinity$  = సాధ్యం కాదు.
*/
var coinChange = function (coins, amount) {
  // Infinity తో fill: "ఇంకా reach అవ్వలేదు" అని అర్థం.
  const dp = new Array(amount + 1).fill(Infinity);
  dp[0] = 0; // base: 0 amount → 0 coins

  for (let a = 1; a <= amount; a++) {
    for (const coin of coins) {
      // coin ని చివరిగా వాడితే:  $dp[a-coin] + 1$ .  $coin \leq a$  అయితేనే valid.
      if (coin <= a && dp[a - coin] + 1 < dp[a]) {
        dp[a] = dp[a - coin] + 1;
      }
    }
  }
}
```

```

}

return dp[amount] === Infinity ? -1 : dp[amount];
};

```

- **Dry Run:** coins = [1,2,5] , amount = 11 . (కొన్ని కీలక values)

a	dp[a] = min over coins (dp[a-coin]+1)	value
0	base	0
1	dp[0]+1 = 1	1
2	min(dp[1]+1, dp[0]+1) = 1	1
5	min(dp[4]+1, dp[3]+1, dp[0]+1) = 1	1
6	min(dp[5]+1, dp[4]+1, dp[1]+1) = 2	2
10	min(dp[9]+1, dp[8]+1, dp[5]+1=2)	2
11	min(dp[10]+1=3, dp[9]+1, dp[6]+1=3)	3

Return 3 (5+5+1). 

- **Complexity:** Time $O(\text{amount} \times \text{coins.length})$ — ప్రతి amount కి అన్ని coins. Space $O(\text{amount})$ dp array.
- **గుర్తుంచుకోవాల్సినది:** $dp[a] = \min_c (dp[a-c] + 1)$, $dp[0]=0$, unreachable = Infinity. "కనిష్ట items తో target కూడబెట్టడం, unlimited supply" = **unbounded knapsack**. Coin Change II ("ఎన్ని ways" — count) కూడా దగ్గరి బంధువు (min బదులు sum). Greedy తప్పు అని గుర్తుంచుకో — ఇదే ఈ problem యొక్క interview trap.
- **సాధారణ తప్పులు:**
 - **Greedy వాడటం** (ఎప్పుడూ largest coin) — కొన్ని inputs కి తప్పు answer. తప్పకుండా DP.
 - **amount = 0** edge: $dp[0] = 0$ కాబట్టి loop నడవదు, సరిగ్గా 0 return.
 - **Infinity overflow:** $dp[a-coin] + 1$ లో $dp[a-coin]$ Infinity అయితే $Infinity + 1 = Infinity$ — JavaScript లో సురక్షితం (కొన్ని భాషల్లో integer overflow). కానీ $dp[a-coin] !== Infinity$ check పెట్టుకోవడం clean.
 - **-1** return మర్చిపోవడం — impossible case ని Infinity గా వదిలేయకూడదు.

5. Longest Increasing Subsequence (LeetCode #300) — Medium

- **సమస్య:** ఒక integer array nums ఇచ్చారు. దానిలో **strictly increasing** subsequence (వరుసలో ఉండనవసరం లేదు, కానీ order maintain చెయ్యాలి) యొక్క **గరిష్ట length** ఎంత?
- **Key constraints:** $1 \leq \text{nums.length} \leq 2500$. (Subsequence = కొన్ని elements ని తీసేసి, మిగిలినవాటి order అలాగే ఉంచడం.)
- **ఉదాహరణ:**

Input: nums = [10,9,2,5,3,7,101,18]

Output: 4

వివరణ: [2,3,7,101] (లేదా [2,3,7,18]) – length 4 increasing subsequence.

• ఎలా ఆలోచించాలి (5-step recipe):

Subsequence problems లో కీలక trick: **subsequence యొక్క చివరి element మీద state ని fix చెయ్యి**. ప్రశ్న: **index i వద్ద ముగిసే longest increasing subsequence ఎంత?** i వద్ద ముగియాలంటే, దాని ముందు ఒక $j < i$ element ఉండాలి ఎక్కడ $nums[j] < nums[i]$ (increasing) – అప్పుడు $dp[i] = dp[j] + 1$. అలాంటి j లన్నిటిలో max తీసుకో. ఏ j లేకపోతే $nums[i]$ ఒక్కటే (length 1).

1. **State:** $dp[i]$ = index i వద్ద **exactly ముగిసే** longest strictly-increasing subsequence యొక్క length.

2. **Recurrence:** $dp[i] = 1 + \max\{ dp[j] : j < i \text{ and } nums[j] < nums[i] \}$ (అలాంటి j ఏదీ లేకపోతే $dp[i] = 1$).

3. **Base case:** ప్రతి $dp[i] = 1$ (కనీసం element తనను తాను ఒక subsequence).

4. **Order:** $i = 0$ నుండి $n-1$; ప్రతి i కి $j = 0$ నుండి $i-1$.

5. **Answer:** $\max(dp[0..n-1])$ – **గమనిక: $dp[n-1]$ కాదు!** LIS ఏ index లోనైనా ముగియవచ్చు, కాబట్టి table లో గరిష్ఠం.

• **Brute Force (recursion):** ప్రతి element కి "తీసుకో / వదిలేయ్" + increasing constraint $\rightarrow O(2^n)$ subsequences. Overlapping subproblems ("ఇక్కడి నుండి ముందుకు LIS" పదే పదే).

• Optimal Approach:

◦ **DP $O(n^2)$:** $dp[i]$ ని nested loop తో (కొద్ది primary solution). $n \leq 2500$ కి సరిపోతుంది.

◦ **Patience sorting $O(n \log n)$:** $tails[k] = \text{length } (k+1) \text{ ఉన్న increasing subsequences అన్నిటిలో అతి చిన్న చివరి element}$. ప్రతి number కి binary search తో సరైన స్థానం. ఇది faster, interview లో "can you do better than $O(n^2)$?" కి జవాబు.

• Solution (JavaScript):

```
/**
 * @param {number[]} nums
 * @return {number} - LIS length
 */
*
*  $dp[i]$  = index  $i$  వద్ద ముగిసే LIS length.
*  $dp[i] = 1 + \max(dp[j])$  for  $j < i$  where  $nums[j] < nums[i]$ .
* Answer =  $\max(dp)$ . ( $dp[n-1]$  కాదు!)
*/
var lengthOfLIS = function (nums) {
  const n = nums.length;
  const dp = new Array(n).fill(1); // ప్రతి element కనీసం length-1 LIS
  let best = 1;

  for (let i = 1; i < n; i++) {
    for (let j = 0; j < i; j++) {
      //  $nums[j]$  చిన్నదైతే,  $j$  వద్ద ముగిసే LIS కి  $nums[i]$  ని జోడించవచ్చు.
      if (nums[j] < nums[i] && dp[j] + 1 > dp[i]) {
        dp[i] = dp[j] + 1;
      }
    }
  }
  return best;
}
```



```

    }
  }
  if (dp[i] > best) best = dp[i]; // table లంతటా max track
}

return best;
};

/* O(n log n) – patience sorting (బీసెస్, faster):
var lengthOfLIS = function (nums) {
  const tails = []; // tails[k] = length (k+1) subseq యొక్క చిన్న tail
  for (const x of nums) {
    let lo = 0, hi = tails.length;
    while (lo < hi) { // x ని ఉంచాల్సిన మొదటి >= స్థానం
      const mid = (lo + hi) >> 1;
      if (tails[mid] < x) lo = mid + 1;
      else hi = mid;
    }
    tails[lo] = x; // replace లేదా extend
  }
  return tails.length;
}; */

```

- **Dry Run ($O(n^2)$):** `nums = [10,9,2,5,3,7,101,18]` .

i	nums[i]	ఏ j లు <code>nums[j]<nums[i]</code> & <code>max dp[j]</code>	dp[i]
0	10	—	1
1	9	—	1
2	2	—	1
3	5	j=2 (2<5), dp=1	2
4	3	j=2 (2<3), dp=1	2
5	7	j=3 (5<7) dp=2, or j=4 dp=2	3
6	101	j=5 (7<101) dp=3	4
7	18	j=5 (7<18) dp=3	4

`max(dp) = 4` .  (subsequence [2,5,7,101] length 4.)

- **Complexity:** $O(n^2)$ DP — Time $O(n^2)$, Space $O(n)$. Patience sorting — Time $O(n \log n)$, Space $O(n)$.
- **గుర్తుంచుకోవాల్సినది:** `dp[i] = 1 + max(dp[j])` for `j<i`, `nums[j]<nums[i]` ; `answer = max(dp)` .
 "Longest/optimal subsequence ending at i" అనే state ఏ subsequence-DP క్లెనా (Longest Chain, Russian Doll Envelopes, Largest Divisible Subset) పునాది. **Answer table లో max — చివరి index కాదు** అనేది ఈ family యొక్క ప్రత్యేకత.
- **సాధారణ తప్పులు:**

- **return dp[n-1]** అని రాయడం – తప్పు! LIS ఏ index లోనైనా ముగియవచ్చు. **max(dp)** కావాలి.
 - **<= vs < :** "strictly increasing" కి **nums[j] < nums[i]** . **<=** వాడితే duplicates ని లెక్కించి over-count.
 - Patience sorting లో tails array **actual subsequence కాదు** – కేవలం length ని ఇస్తుంది (tails యొక్క content ఒక valid LIS కాకపోవచ్చు). దీన్ని గుర్తుంచుకో.
-

Multidimensional (2D) DP

1D DP లో state ఒక్క index ($dp[i]$). కానీ కొన్ని problems లో subproblem ని describe చెయ్యడానికి **రెండు parameters** అవసరం — grid లో (row, col), లేదా రెండు strings యొక్క (i, j) prefixes. అప్పుడు $dp[i][j]$ అనే **2D table** వాడతాం. Same 5-step recipe — కేవలం state కి రెండు dimensions.

Pattern: 2D DP (Grid DP + String DP)

వివరణ

2D DP రెండు రూపాల్లో వస్తుంది:

(1) **Grid DP** — ఒక 2D grid/matrix ఇచ్చి, ఒక మూల నుండి మరో మూలకి path/count/optimal అడుగుతారు. $dp[i][j]$ = cell (i, j) కి సంబంధించిన answer (ఉదా: ఇక్కడికి చేరే కనిష్ట cost, ఇక్కడ ముగిసే largest square). సాధారణంగా ఒక cell దాని **పైన** ($dp[i-1][j]$), **ఎడమ** ($dp[i][j-1]$), **diagonal** ($dp[i-1][j-1]$) neighbours మీద ఆధారపడుతుంది.

(2) **String DP** — రెండు strings (లేదా ఒక string యొక్క రెండు indices) మధ్య relation. $dp[i][j]$ = మొదటి string యొక్క మొదటి **i** అక్షరాలు మరియు రెండో string యొక్క మొదటి **j** అక్షరాల మధ్య answer (ఉదా: edit distance, interleaving, LCS). ఇక్కడ ఒక అదనపు trick: **empty prefix కోసం extra row/column** ($dp[0][*]$, $dp[*][0]$) — అందుకే matrix size $(m+1) \times (n+1)$.

Real-life analogy: ఒక city road-grid ని ఊహించు. నువ్వు top-left నుండి bottom-right కి వెళ్ళాలి, కుడికి లేదా కిందకి మాత్రమే. ప్రతి junction (i, j) దగ్గర "ఇక్కడికి చేరడానికి కనిష్ట cost ఎంత?" అనేది — నీకు **పైనుండి వచ్చిన cost** మరియు **ఎడమనుండి వచ్చిన cost** లో చిన్నదానికి, ఈ junction cost కలిపితే వస్తుంది. ప్రతి junction ని ఒకసారే లెక్కించి board (dp table) మీద రాసుకుంటూ, top-left నుండి bottom-right దాకా నింపుకుంటూ వెళ్తావు. అదే grid DP.

ఎలా గుర్తించాలి (recognition signals)

- Grid/matrix** ఇచ్చి "top-left $\rightarrow$ bottom-right", "కనిష్ట path", "ఎన్ని paths", "largest square/rectangle" అని అడిగితే.
- రెండు strings/sequences** ని align/compare/transform చెయ్యాలంటే (edit distance, LCS, interleaving, matching).
- ఒక్క index తో subproblem ని describe చెయ్యలేకపోతే — "ఇక్కడ నా position రెండు విషయాలపై ఆధారపడుతోంది" అనిపిస్తే $\rightarrow$ రెండో dimension కావాలి.
- Palindrome substrings — $dp[i][j]$ = "s[i..j] palindrome?" (రెండు ends).

dp[i][j] meaning + Filling Order (అత్యంత కీలకం)

2D DP లో **fill order** తప్పిస్తే garbage వస్తుంది. సూత్రం: **dp[i][j]** ని లెక్కించే ముందు, అది ఆధారపడే అన్ని cells ఇప్పటికే నింపబడి ఉండాలి.

- dp[i][j] పైన/ఎడమ/diagonal (dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) మీద ఆధారపడితే → **top-to-bottom, left-to-right** నింపు (i పెరుగుతూ, j పెరుగుతూ). ఇది అత్యంత common (Grid DP, Edit Distance, Maximal Square).
- dp[i][j] కింద/కుడి (dp[i+1][j], dp[i][j+1]) మీద ఆధారపడితే → **bottom-to-top** నింపు (Triangle bottom-up).
- Substring DP (dp[i][j] depends on dp[i+1][j-1] — inner substring) → **i తగ్గుతూ (high→low), j పెరుగుతూ**, లేదా length పెరిగే క్రమంలో (Longest Palindromic Substring).

Template (Grid DP + String DP)

```
// Grid DP (top-left → bottom-right):
const m = grid.length, n = grid[0].length;
const dp = Array.from({ length: m }, () => new Array(n).fill(0));
dp[0][0] = /* base */;
// మొదటి row & column ని విడిగా నింపు (వాటికి ఒకే neighbour)
for (let j = 1; j < n; j++) dp[0][j] = /* dp[0][j-1] నుండి */;
for (let i = 1; i < m; i++) dp[i][0] = /* dp[i-1][0] నుండి */;
for (let i = 1; i < m; i++)
  for (let j = 1; j < n; j++)
    dp[i][j] = /* dp[i-1][j], dp[i][j-1] ... ని combine */;
return dp[m - 1][n - 1];

// String DP (రెండు strings; empty prefix కోసం +1 padding):
const dp = Array.from({ length: m + 1 }, () => new Array(n + 1).fill(0));
for (let i = 0; i <= m; i++) dp[i][0] = /* base: s1[0..i] vs empty */;
for (let j = 0; j <= n; j++) dp[0][j] = /* base: empty vs s2[0..j] */;
for (let i = 1; i <= m; i++)
  for (let j = 1; j <= n; j++)
    // s1[i-1], s2[j-1] గమనించు — 1-offset (dp index i ↔ char i-1)
    dp[i][j] = /* recurrence */;
return dp[m][n];
```

Complexity

2D DP సాధారణంగా **Time $O(m \cdot n)$** (ప్రతి cell ఒక్కసారే, $O(1)$ work), **Space $O(m \cdot n)$** . చాలా సందర్భాల్లో dp[i][j] కేవలం **previous row** (dp[i-1]) మీద ఆధారపడితే, పూర్తి matrix కాకుండా ఒక 1D row ఉంచి **Space $O(n)$** కి తగ్గించవచ్చు (rolling row).

గుర్తుంచుకో: 2D DP = 1D DP కి రెండే dimension. **State ని రెండు parameters తో define చేసి, fill order జాగ్రత్తగా నిర్ణయిస్తే** చాలు. String DP లో $(m+1) \times (n+1)$ padding + $s[i-1]$ offset — ఈ రెండూ మర్చిపోకు. కింది 9 problems ఇదే pattern యొక్క variations.

6. Triangle (LeetCode #120) – Medium

- **సమస్య:** ఒక triangle (rows array) ఇచ్చారు – row i లో $i+1$ elements. పైనుండి కిందకి, ప్రతి అడుగులో **పక్కపక్క (adjacent) number** కి మాత్రమే వెళ్ళగలవు (index j నుండి j లేదా $j+1$ కి). Top నుండి bottom కి చేరే **కనిష్ఠ path sum** ఎంత?
- **Key constraints:** $1 \leq \text{triangle.length} \leq 200$. Follow-up: $O(n)$ extra space లో చెయ్యగలవా?
- **ఉదాహరణ:**

```
Input: [[2], [3,4], [6,5,7], [4,1,8,3]]
Output: 11
వివరణ: 2 → 3 → 5 → 1 = 11 (కనిష్ఠ path).
      2
     3 4
    6 5 7
   4 1 8 3
```

- **ఎలా ఆలోచించాలి (5-step recipe):**

Top-down గా ఆలోచిస్తే "ఏ adjacent కి వెళ్ళాలి" అనే branching కష్టం. **Bottom-up** గా తిరగేస్తే అందంగా అవుతుంది: చివరి row నుండి పైకి. కీలక ప్రశ్న: **cell (i, j) నుండి bottom కి చేరే కనిష్ఠ path sum ఎంత?** cell (i, j) నుండి కింద (i+1, j) లేదా (i+1, j+1) కి వెళ్ళగలం – ఆ రెండిట్లో చిన్నదానికి ఈ cell విలువ కలిపితే వస్తుంది.

1. **State:** $dp[j]$ = ప్రస్తుత row యొక్క cell j నుండి **bottom వరకు** కనిష్ఠ path sum (bottom-up, 1D rolling).
 2. **Recurrence:** $dp[j] = \text{triangle}[i][j] + \min(dp[j], dp[j+1])$ – కింది row (ఇప్పటికే dp లో ఉన్న) నుండి రెండు choices లో min.
 3. **Base case:** చివరి row $\rightarrow dp[j] = \text{triangle}[\text{last}][j]$ (అక్కడే bottom).
 4. **Order:** i = second-last row నుండి row 0 వరకు (కింది row ముందే dp లో ఉండాలి).
 5. **Answer:** $dp[0]$ (apex నుండి కనిష్ఠ path).
- **Brute Force (recursion):** ప్రతి cell నుండి రెండు దారులు $\rightarrow O(2^n)$ paths (Pascal-triangle పరిమాణంలో). Cell (i,j) పదే పదే different పైనుండి = overlapping.
 - **Optimal Approach:**
 - **Memoization:** $\text{minPath}(i, j)$ కి 2D memo $\rightarrow O(n^2)$.
 - **Tabulation (bottom-up):** పూర్తి 2D dp $\rightarrow O(n^2)$ space; కానీ ఒక row మాత్రమే అవసరం కాబట్టి **1D array** చాలు $\rightarrow O(n)$ space (follow-up కి జవాబు). చివరి row ని copy చేసి, పైకి నింపు.

- **Solution (JavaScript):**

```
/**
 * @param {number[][]} triangle
 * @return {number} - top → bottom కనిష్ఠ path sum
 *
 * Bottom-up: dp[j] = triangle[i][j] + min(dp[j], dp[j+1]).
 * చివరి row ను seed చేసి పైకి నింపు. 1D array చాలు (O(n) space).
 */
```

```

var minimumTotal = function (triangle) {
  const n = triangle.length;
  const dp = [...triangle[n - 1]]; // చివరి row ని copy (base case)

  // Second-last row నుండి apex దాకా పైకి
  for (let i = n - 2; i >= 0; i--) {
    for (let j = 0; j <= i; j++) {
      // కింది row లో adjacent రెండు (dp[j], dp[j+1]) లో చిన్నది + current
      dp[j] = triangle[i][j] + Math.min(dp[j], dp[j + 1]);
    }
  }

  return dp[0]; // apex వద్ద answer
};

```

- **Dry Run:** `[[2], [3,4], [6,5,7], [4,1,8,3]]` . dp seed = last row `[4,1,8,3]` .

i (row)	j loop: <code>dp[j] = t[i][j] + min(dp[j], dp[j+1])</code>	dp after
3 (base)	—	[4, 1, 8, 3]
2 [6,5,7]	6+min(4,1)=7; 5+min(1,8)=6; 7+min(8,3)=10	[7, 6, 10, 3]
1 [3,4]	3+min(7,6)=9; 4+min(6,10)=10	[9, 10, 10, 3]
0 [2]	2+min(9,10)=11	[11, 10, 10, 3]

Return `dp[0]` = 11 . 

- **Complexity:** Time $O(n^2)$ — triangle లో మొత్తం cells సంఖ్య ($\sim n^2/2$). Space $O(n)$ — ఒక్క row (bottom-up rolling). Follow-up satisfied.
- **గుర్తుంచుకోవాల్సినది: Bottom-up:** `dp[j] = triangle[i][j] + min(dp[j], dp[j+1])` . "Triangle/grid లో adjacent కి మాత్రమే కదలగలిగే path" ని కింది నుండి పైకి solve చేస్తే branching సులభం అవుతుంది. Top-down కష్టమైతే bottom-up ప్రయత్నించడం ఒక powerful mental switch.
- **సాధారణ తప్పులు:**
 - **Top-down గా బలవంతంగా చెయ్యడం** — first-row nesting, boundary handling కష్టం. Bottom-up cleaner.
 - **1D dp లో `dp[j]` , `dp[j+1]` ఒకే iteration లో overwrite:** bottom-up left→right లో ఇది సురక్షితం (`dp[j]` ని update చేసేటప్పుడు `dp[j+1]` ఇంకా పాత/కింది row value). ఈ order ని మార్చకు.
 - **min** బదులు తప్పుగా `Math.min(dp[j-1], dp[j])` (ఎడమ) వాడటం — ఇది adjacency తప్పు. కింది row లో j మరియు j+1 మాత్రమే reachable.

7. Minimum Path Sum (LeetCode #64) — Medium

- **సమస్య:** ఒక $m \times n$ grid లో non-negative numbers ఉన్నాయి. Top-left `(0,0)` నుండి bottom-right `(m-1,n-1)` కి వెళ్ళాలి, కుడికి లేదా కిందకి మాత్రమే కదలగలవు. Path మీది numbers sum కనిష్ఠం ఎంత?

- **Key constraints:** $1 \leq m, n \leq 200$, $0 \leq \text{grid}[i][j] \leq 200$.

- **ఉదాహరణ:**

Input: `grid = [[1,3,1],
 [1,5,1],
 [4,2,1]]`

Output: 7

వివరణ: $1 \rightarrow 3 \rightarrow 1 \rightarrow 1 \rightarrow 1 = 7$ (కుడి, కుడి, కింద, కింద).

- **ఎలా ఆలోచించాలి (5-step recipe):**

Classic **Grid DP**. కీలక ప్రశ్న: cell (i, j) కి చేరే కనిష్ఠ path sum ఎంత? (i, j) కి రావాలంటే — దాని పైననుండి $(i-1, j)$ లేదా ఎడమనుండి $(i, j-1)$ మాత్రమే రాగలం (కుడి/కింద కదలిక కాబట్టి). ఆ రెండిల్లో చిన్నదానికి ఈ cell విలువ కలిపితే వస్తుంది.

1. **State:** $\text{dp}[i][j]$ = $(0,0)$ నుండి (i, j) కి చేరే కనిష్ఠ path sum.
2. **Recurrence:** $\text{dp}[i][j] = \text{grid}[i][j] + \min(\text{dp}[i-1][j], \text{dp}[i][j-1])$.
3. **Base case:** $\text{dp}[0][0] = \text{grid}[0][0]$. మొదటి row: $\text{dp}[0][j] = \text{dp}[0][j-1] + \text{grid}[0][j]$ (ఎడమనుండి మాత్రమే). మొదటి column: $\text{dp}[i][0] = \text{dp}[i-1][0] + \text{grid}[i][0]$ (పైననుండి మాత్రమే).
4. **Order:** top-to-bottom, left-to-right (పైన & ఎడమ ముందే ready).
5. **Answer:** $\text{dp}[m-1][n-1]$.

- **Brute Force (recursion):** ప్రతి cell నుండి కుడి/కింద రెండు దారులు $\rightarrow$ **exponential**. Cell (i,j) పదే పదే వేర్వేరు paths ద్వారా = overlapping.

- **Optimal Approach:**

- **Memoization:** `minSum(i, j)` + 2D memo $\rightarrow O(m \cdot n)$.
- **Tabulation:** 2D dp fill $\rightarrow O(m \cdot n)$ time & space. Space-optimize: $\text{dp}[i][j]$ కేవలం current row + పై row మీద ఆధారపడుతుంది కాబట్టి **1D row** ($O(n)$ space) చాలు.

- **Solution (JavaScript):**

```
/**
 * @param {number[][]} grid
 * @return {number} - top-left  $\rightarrow$  bottom-right కనిష్ఠ path sum
 *
 *  $\text{dp}[i][j] = \text{grid}[i][j] + \min(\text{పైన } \text{dp}[i-1][j], \text{ఎడమ } \text{dp}[i][j-1])$ .
 * (స్వచ్ఛత కోసం పూర్తి 2D; 1D optimization కింద గమనికల్.)
 */
var minPathSum = function (grid) {
    const m = grid.length, n = grid[0].length;
    const dp = Array.from({ length: m }, () => new Array(n).fill(0));

    dp[0][0] = grid[0][0]; // base

    // మొదటి row: ఎడమనుండి మాత్రమే రాగలం
    for (let j = 1; j < n; j++) dp[0][j] = dp[0][j - 1] + grid[0][j];
    // మొదటి column: పైననుండి మాత్రమే రాగలం
    for (let i = 1; i < m; i++) dp[i][0] = dp[i - 1][0] + grid[i][0];
```

```
// మిగతా cells: పైన లేదా ఎడమ లో చిన్నది
for (let i = 1; i < m; i++) {
  for (let j = 1; j < n; j++) {
    dp[i][j] = grid[i][j] + Math.min(dp[i - 1][j], dp[i][j - 1]);
  }
}

return dp[m - 1][n - 1];
};

/* Space O(n) - 1D rolling row:
var minPathSum = function (grid) {
  const m = grid.length, n = grid[0].length;
  const dp = new Array(n).fill(Infinity);
  dp[0] = 0;
  for (let i = 0; i < m; i++)
    for (let j = 0; j < n; j++)
      dp[j] = grid[i][j] + (j === 0 ? dp[j] : Math.min(dp[j], dp[j - 1]));
  return dp[n - 1];
}; */
```

- **Dry Run:** grid `[[1,3,1],[1,5,1],[4,2,1]]` . dp table నింపుతూ:

dp	col0	col1	col2
row0	1	1+3=4	4+1=5
row1	1+1=2	5+min(4,2)=7	1+min(5,7)=6
row2	2+4=6	2+min(7,6)=8	1+min(6,8)=7

Return `dp[2][2] = 7` . 

- **Complexity:** Time $O(m \cdot n)$ — ప్రతి cell ఒక్కసారి. Space $O(m \cdot n)$ (2D) లేదా $O(n)$ (1D rolling row).
- **గుర్తుంచుకోవాల్సినది:** `dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])` . ఇది Grid DP యొక్క mother template — "top-left నుండి bottom-right, కుడి/కింద మాత్రమే" అనే ప్రతి problem (Unique Paths, Dungeon Game, Cherry Pickup కూడా variations). **మొదటి row/column ని విడిగా seed చెయ్యడం** ఈ pattern యొక్క గుండె.
- **సాధారణ తప్పులు:**
 - **మొదటి row/column ని initialize చెయ్యకపోవడం** — వాటికి పైన/ఎడమ neighbour లేదు; సరిగ్గా cumulative sum గా seed చెయ్యాలి. లేకపోతే `min(undefined, ...)` → NaN.
 - `Math.min(dp[i-1][j], dp[i][j-1])` లో indices తారుమారు (`dp[i][j-1]` బదులు `dp[j][i-1]`) — off-by-index bug.
 - Grid ని **modify in-place** చేసి interviewer input mutate చెయ్యడం (కొన్ని optimized solutions grid నే dp గా వాడతాయి) — అడగకుండా చేయకు.

8. Unique Paths II (LeetCode #63) – Medium

- **సమస్య:** ఒక $m \times n$ grid లో robot top-left లో ఉంది, bottom-right కి వెళ్ళాలి (కుడి/కింద మాత్రమే). Grid లో కొన్ని cells లో **obstacles** (1) ఉన్నాయి – వాటిమీదుగా వెళ్ళలేం. 0 = ఖాళీ. Bottom-right కి చేరే **distinct paths** సంఖ్య ఎంత?
- **Key constraints:** $1 \leq m, n \leq 100$. Cells 0 లేదా 1. Start లేదా end obstacle అయితే answer 0.
- **ఉదాహరణ:**

Input: $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

Output: 2

వివరణ: మధ్యలో obstacle. రెండు దారులు: కుడి-కుడి-కింద-కింద, కింద-కింద-కుడి-కుడి.

- **ఎలా ఆలోచించాలి (5-step recipe):**

ఇది **count DP** (ఎన్ని ways) + Grid DP. Min Path Sum కి దగ్గరి బంధువు – కానీ ఇక్కడ **min** బదులు **sum** (ways add అవుతాయి), మరియు obstacle అనే twist. కీలక ప్రశ్న: **cell (i, j) కి చేరే distinct paths ఎన్ని?** (i, j) obstacle అయితే 0 (ఇక్కడికి రాలేం). లేకపోతే = **పైననుండి వచ్చిన ways + ఎడమనుండి వచ్చిన ways** = $dp[i-1][j] + dp[i][j-1]$.

1. **State:** $dp[i][j]$ = (0,0) నుండి (i,j) కి చేరే distinct paths సంఖ్య (obstacles ని తప్పించుకుంటూ).
 2. **Recurrence:** $grid[i][j] == 1$ అయితే $dp[i][j] = 0$; లేకపోతే $dp[i][j] = dp[i-1][j] + dp[i][j-1]$.
 3. **Base case:** $dp[0][0] = (grid[0][0] == 1) ? 0 : 1$ (start obstacle అయితే ఏ path లేదు).
 4. **Order:** top-to-bottom, left-to-right.
 5. **Answer:** $dp[m-1][n-1]$.
- **Brute Force (recursion):** ప్రతి cell నుండి కుడి/కింద, obstacle hit అయితే dead-end → **exponential**. Cell (i,j) overlapping.
 - **Optimal Approach:**
 - **Memoization:** $paths(i, j)$ + 2D memo → $O(m \cdot n)$.
 - **Tabulation:** 2D dp; obstacle cell = 0, మిగతా = పైన + ఎడమ → $O(m \cdot n)$. **1D rolling row** తో $O(n)$ space (కింది solution – ఇదే elegant).
 - **Solution (JavaScript):**

```
/**
 * @param {number[][]} obstacleGrid
 * @return {number} - obstacles తప్పించుకుంటూ distinct paths
 */
* dp[j] = obstacle అయితే 0, లేకపోతే dp[j] (పైన) + dp[j-1] (ఎడమ).
* 1D rolling row: dp[j] update అయ్యే ముందు = పై row value.
*/
var uniquePathsWithObstacles = function (obstacleGrid) {
  const m = obstacleGrid.length, n = obstacleGrid[0].length;
```

```

const dp = new Array(n).fill(0);
// Start obstacle అయితే 0 ways, లేకపోతే 1 (base)
dp[0] = obstacleGrid[0][0] === 1 ? 0 : 1;

for (let i = 0; i < m; i++) {
  for (let j = 0; j < n; j++) {
    if (obstacleGrid[i][j] === 1) {
      dp[j] = 0; // obstacle → ఇక్కడికి 0 paths
    } else if (j > 0) {
      // update ముందు dp[j] = పై row (i-1,j); dp[j-1] = ఎడమ (i,j-1)
      dp[j] += dp[j - 1];
    }
    // j === 0 & ఖాళీ: dp[0] పై row నుండి carry అవుతుంది (మార్చవద్దు)
  }
}

return dp[n - 1];
};

```

1D trick ఎలా పని చేస్తుంది: $dp[j]$ ని update చేసే ముందు అది పై row యొక్క $(i-1, j)$ value ని పట్టుకుని ఉంటుంది (గత iteration నుండి). $dp[j-1]$ ఇప్పటికే ఈ row యొక్క $(i, j-1)$ గా update అయింది. కాబట్టి $dp[j] += dp[j-1]$ = పైన + ఎడమ. అందంగా.

- **Dry Run:** $[[0,0,0], [0,1,0], [0,0,0]]$. dp row-by-row (j: 0,1,2):

after row i	dp[0]	dp[1]	dp[2]
init	1	0	0
i=0 [0,0,0]	1	0+1=1	0+1=1
i=1 [0,1,0]	1	obstacle→0	1+0=1
i=2 [0,0,0]	1	1+0=1	1+1=2

Return $dp[2] = 2$. ✓

- **Complexity:** Time $O(m \cdot n)$ — ప్రతి cell ఒక్కసారే. Space $O(n)$ — 1D rolling row.
- **గుర్తుంచుకోవాల్సినది:** $dp[i][j] = dp[i-1][j] + dp[i][j-1]$ (obstacle అయితే 0). Count-paths grid DP = **sum of top + left** (Min Path Sum లో min ఉంటే, ఇక్కడ sum). Obstacle = ఆ state ని 0 చెయ్యడం. ఇదే idea తో ఏ "forbidden cells తో path count" అయినా.
- **సాధారణ తప్పులు:**
 - $dp[0]$ ని ఎప్పుడూ 1 గా init చెయ్యడం — start cell obstacle అయితే 0 కావాలి. లేకపోతే తప్పు count.
 - **Obstacle cell కి** $dp[j]$ ని 0 చెయ్యడం మర్చిపోవడం — obstacle మీదుగా ways leak అవుతాయి.
 - **First column obstacle handling:** ఒక column లో obstacle వచ్చాక, దాని కింది cells కి (ఆ column లో) paths 0 కావాలి — 1D code లో obstacle $dp[j]=0$ చేసి, $j=0$ కి carry ఆగడం వల్ల ఇది సరిగ్గా జరుగుతుంది.

- Integer overflow — పెద్ద grids లో path counts చాలా పెద్దవి కావచ్చు; JavaScript numbers 2^{53} దాకా safe, కానీ constraints ($m, n \leq 100$) లో fine.

9. Longest Palindromic Substring (LeetCode #5) — Medium

- సమస్య:** ఒక string s ఇచ్చారు. దానిలోని **అత్యంత పొడవైన palindromic substring** (ముందు నుండి, వెనుక నుండి ఒకేలా చదివే contiguous భాగం) ని return చెయ్యి.
- Key constraints:** $1 \leq s.length \leq 1000$. **Substring** = contiguous (subsequence కాదు!).
- ఉదాహరణ:**

```
Input: s = "babad"
Output: "bab" ("aba" కూడా valid - ఏదైనా ఒకటి).

Input: s = "cbbd"
Output: "bb"
```

- ఎలా ఆలోచించాలి (5-step recipe):**

Substring problems కి 2D DP idea: **రెండు ends (i, j) మీద state**. కీలక ప్రశ్న: $s[i..j]$ ఒక palindrome అవునా? ఒక string palindrome అవ్వాలంటే — (1) దాని **మొదటి & చివరి అక్షరాలు సమానం** ($s[i] == s[j]$), మరియు (2) **లోపలి భాగం $s[i+1..j-1]$ కూడా palindrome**. ఇదే recurrence! (గుడ్డు లోపల గుడ్డు నమూనా — palindrome లోపల palindrome.)

- State:** $dp[i][j] = \text{true}$ అయితే substring $s[i..j]$ (i, j రెండూ inclusive) ఒక palindrome.
- Recurrence:** $dp[i][j] = (s[i] == s[j]) \text{ AND } (j - i < 3 \text{ OR } dp[i+1][j-1])$. ($j - i < 3$ అంటే $\text{length} \leq 3$ — length 1,2,3 కి లోపలి భాగం trivially palindrome/empty.)
- Base case:** $dp[i][i] = \text{true}$ (ఒక్క అక్షరం ఎప్పుడూ palindrome).
- Order:** $dp[i][j]$ లోపలి $dp[i+1][j-1]$ మీద ఆధారపడుతుంది $\rightarrow i$ ని కింది నుండి పైకి ($n-1 \rightarrow 0$), j ని $i+1$ నుండి పైకి. ఇలా చేస్తే $dp[i+1][*]$ ఇప్పటికే ready.
- Answer:** గరిష్ఠ length ఉన్న (i, j) (start & maxLen track చేస్తూ), చివరికి $s.\text{substring}(\text{start}, \text{start}+\text{maxLen})$.

- Brute Force (recursion):** అన్ని $O(n^2)$ substrings, ప్రతిదానికి palindrome check $O(n) \rightarrow O(n^3)$. Palindrome checks overlap (పెద్ద substring check లో చిన్నవి పడే పడే).
- Optimal Approach:**
 - 2D DP:** $dp[i][j]$ table $\rightarrow O(n^2)$ time & space. (కింది primary solution.)
 - Expand-around-center:** ప్రతి center (n single + $n-1$ double = $2n-1$ centers) నుండి బయటకి విస్తరించు $\rightarrow O(n^2)$ time, $O(1)$ space — DP కంటే space తక్కువ, చాలా intuitive. (Bonus గా కింద.)
 - (Manacher's $O(n)$ ఉంది కానీ interview లో అరుదు.)
- Solution (JavaScript):**

```

/**
 * @param {string} s
 * @return {string} - longest palindromic substring
 *
 * dp[i][j] = s[i..j] palindrome.
 * dp[i][j] = s[i]==s[j] && (j-i<3 || dp[i+1][j-1]).
 * i ని కింది నుండి పైకి (inner substring ముందే ready అవ్వాలి).
 */
var longestPalindrome = function (s) {
  const n = s.length;
  if (n < 2) return s; // ఒక్క అక్షరం/ఖాళీ → అదే palindrome

  const dp = Array.from({ length: n }, () => new Array(n).fill(false));
  let start = 0, maxlen = 1;

  for (let i = 0; i < n; i++) dp[i][i] = true; // length-1 base

  // i ని పెద్ద index నుండి చిన్నదానికి (dp[i+1][j-1] ముందే కావాలి)
  for (let i = n - 1; i >= 0; i--) {
    for (let j = i + 1; j < n; j++) {
      if (s[i] === s[j]) {
        // length 2,3 (j-i===1||2) → లోపల trivial; లేకపోతే dp[i+1][j-1]
        if (j - i === 1 || dp[i + 1][j - 1]) {
          dp[i][j] = true;
          if (j - i + 1 > maxlen) {
            start = i;
            maxlen = j - i + 1;
          }
        }
      }
    }
  }

  return s.substring(start, start + maxlen);
};

/* Expand-around-center - O(n^2) time, O(1) space (bonus):
var longestPalindrome = function (s) {
  let start = 0, maxlen = 1;
  const expand = (l, r) => {
    while (l >= 0 && r < s.length && s[l] === s[r]) { l--; r++; }
    if (r - l - 1 > maxlen) { start = l + 1; maxlen = r - l - 1; }
  };
  for (let i = 0; i < s.length; i++) {
    expand(i, i); // odd length center
    expand(i, i + 1); // even length center
  }
  return s.substring(start, start + maxlen);
}; */

```

- **Dry Run:** s = "babad" (indices b0 a1 b2 a3 d4). క్రింద dp[i][j] :

dp	j=0	j=1	j=2	j=3	j=4
i=0 (b)	T	F	T (s0==s2, len3)	F	F
i=1 (a)		T	F	T (s1==s3, len3)	F
i=2 (b)			T	F	F
i=3 (a)				T	F
i=4 (d)					T

dp[0][2]=true (len 3, "bab") → start=0, maxlen=3. Return "bab" . ✓

- **Complexity:** DP — Time $O(n^2)$, Space $O(n^2)$. Expand-around-center — Time $O(n^2)$, Space $O(1)$ (interview లో ఇది prefer చెయ్యవచ్చు).
- **గుర్తుంచుకోవాల్సినది:** $dp[i][j] = s[i]==s[j] \ \&\& \ (j-i<3 \ || \ dp[i+1][j-1])$. "Palindrome లోపల palindrome" — outer chars match + inner palindrome. **Fill order:** i పైకి (n-1→0), j కుడికి (inner substring dependency). ఈ interval-DP fill order (Palindromic Substrings count, Longest Palindromic Subsequence కి కూడా) చాలా ముఖ్యం.
- **సాధారణ తప్పులు:**
 - **Fill order తప్పు:** i ని 0 నుండి పైకి (సాధారణ order) నింపితే dp[i+1][j-1] ఇంకా compute కాలేదు → garbage. i తప్పకుండా n-1 నుండి కిందకి.
 - $j - i < 3$ **shortcut మర్చిపోవడం:** length 2 ("bb") కి dp[i+1][j-1] = dp[i+1][i] అంటే i+1 > i-1... empty/invalid — దాన్ని $j-i<3$ తో short-circuit చెయ్యాలి. లేకపోతే length-2 palindromes miss.
 - **Substring vs subsequence గందరగోళం** — ఇది contiguous substring. Longest Palindromic *Subsequence* (#516) వేరే problem (వేరే recurrence).

10. Interleaving String (LeetCode #97) — Medium

- **సమస్య:** మూడు strings $s1$, $s2$, $s3$ ఇచ్చారు. $s3$ అనేది $s1$ మరియు $s2$ ని **interleave** (కలగలపడం — ఒక్కో string యొక్క అక్షరాల **relative order maintain** చేస్తూ, రెండింటినీ కలిపి) చేసి తయారు చెయ్యవచ్చా? **true / false** .
- **Key constraints:** $0 \leq s1.length, s2.length \leq 100$. Interleave అవ్వాలంటే $s1.length + s2.length == s3.length$ తప్పనిసరి.
- **ఉదాహరణ:**

Input: $s1="aabcc"$, $s2="dbbca"$, $s3="aadbbcbcac"$
Output: **true** (a a | dbbc | b | c | a c ... $s1, s2$ order maintain అవుతూ)

Input: $s1="aabcc"$, $s2="dbbca"$, $s3="aadbbbacc"$
Output: **false**

- ఎలా ఆలోచించాలి (5-step recipe):

రెండు strings ని consume చేస్తున్నాం కాబట్టి **రెండు pointers = 2D state**. కీలక ప్రశ్న: **s1** యొక్క మొదటి **i** అక్షరాలు + **s2** యొక్క మొదటి **j** అక్షరాలు కలిపి, **s3** యొక్క మొదటి **i+j** అక్షరాలు తయారు చెయ్యగలమా? **s3** లో తర్వాతి అక్షరం (**s3[i+j-1]**) ఎక్కడనుండి వచ్చింది? రెండు అవకాశాలు –

- **s1 నుండి:** **s1[i-1] == s3[i+j-1]** మరియు మిగతా (**i-1, j**) ఇంతకుముందు **valid (dp[i-1][j])**.
- **s2 నుండి:** **s2[j-1] == s3[i+j-1]** మరియు **dp[i][j-1]** **valid**. ఏదో ఒకటి పని చేస్తే చాలు (OR).

1. **State:** **dp[i][j] = true** అయితే **s1[0..i-1] + s2[0..j-1]** interleave చేసి **s3[0..i+j-1]** తయారు చెయ్యవచ్చు.
2. **Recurrence:** **dp[i][j] = (dp[i-1][j] && s1[i-1]==s3[i+j-1]) OR (dp[i][j-1] && s2[j-1]==s3[i+j-1])**.
3. **Base case:** **dp[0][0] = true** (ఖాళీ + ఖాళీ = ఖాళీ). మొదటి column (**s2** వాడకుండా): **dp[i][0] = dp[i-1][0] && s1[i-1]==s3[i-1]**. మొదటి row similarly **s2** తో.
4. **Order:** top-to-bottom, left-to-right.
5. **Answer:** **dp[m][n]** (**m=s1.length, n=s2.length**). ముందు **m+n != s3.length** అయితే వెంటనే **false**.

- **Brute Force (recursion):** ప్రతి **s3** అక్షరానికి "s1 నుండి తీసుకో / s2 నుండి తీసుకో" → **O(2^{m+n})**. State (i, j) పదే పదే వేర్వేరు paths ద్వారా = overlapping.

- **Optimal Approach:**

- **Memoization:** **can(i, j)** + 2D memo → **O(m·n)**.
- **Tabulation:** **(m+1)×(n+1)** dp → **O(m·n)** time & space. 1D row తో **O(n)** space సాధ్యం.

- **Solution (JavaScript):**

```
/**
 * @param {string} s1
 * @param {string} s2
 * @param {string} s3
 * @return {boolean}
 */
*
* dp[i][j] = s1[0..i] + s2[0..j] → s3[0..i+j] తయారు చెయ్యవచ్చు.
* s3 లో current char index k = i+j-1.
*/
var isInterleave = function (s1, s2, s3) {
  const m = s1.length, n = s2.length;
  if (m + n !== s3.length) return false; // length match లేకపోతే అసాధ్యం

  const dp = Array.from({ length: m + 1 }, () => new Array(n + 1).fill(false));
  dp[0][0] = true; // base: అన్నీ ఖాళీ

  // మొదటి column: s2 వాడకుండా s1 తోనే s3 prefix
  for (let i = 1; i <= m; i++) {
    dp[i][0] = dp[i - 1][0] && s1[i - 1] === s3[i - 1];
  }

  // మొదటి row: s1 వాడకుండా s2 తోనే
  for (let j = 1; j <= n; j++) {
    dp[0][j] = dp[0][j - 1] && s2[j - 1] === s3[j - 1];
  }
}
```

```

}

for (let i = 1; i <= m; i++) {
  for (let j = 1; j <= n; j++) {
    const k = i + j - 1; // s3 లో ప్రస్తుత అక్షరం
    dp[i][j] =
      (dp[i - 1][j] && s1[i - 1] === s3[k]) || // s1 నుండి తీసుకున్నాం
      (dp[i][j - 1] && s2[j - 1] === s3[k]); // s2 నుండి తీసుకున్నాం
  }
}

return dp[m][n];
};

```

- **Dry Run:** s1="aabcc" (m=5), s2="dbbca" (n=5), s3="aadbcbcbac" . length 5+5=10 ✓. కొన్ని కీలక steps: dp[0][0]=T ; dp[1][0] : s1[0]='a'==s3[0]='a' → T; dp[2][0] : s1[1]='a'==s3[1]='a' → T; dp[2][1] : (dp[1][1]&&...) or (dp[2][0]&&s2[0]='d'==s3[2]='d') → T. ఇలా table నిండి చివరికి dp[5][5] = true . ✓ (రెండో ఉదాహరణలో ఏ path కూడా చివరిదాకా చేరదు → dp[5][5]=false .)
- **Complexity:** Time $O(m \cdot n)$ – ప్రతి (i,j) cell ఒక్కసారే. Space $O(m \cdot n)$ (2D) లేదా $O(n)$ (1D rolling row).
- **గుర్తుంచుకోవాల్సినది:** $dp[i][j] = (dp[i-1][j] \&\& s1[i-1]===s3[i+j-1]) \mid (dp[i][j-1] \&\& s2[j-1]===s3[i+j-1])$. "రెండు sequences ని ఒక target గా merge/interleave చెయ్యగలమా" అనే ప్రతి problem కీ ఈ 2D boolean DP. **s3 index = i+j-1** అనే link ఈ problem యొక్క ఆత్మ.
- **సాధారణ తప్పులు:**
 - **Length check మర్చిపోవడం** – $m + n \neq s3.length$ అయితే వెంటనే false. లేకపోతే index out-of-bounds లేదా తప్పు answer.
 - **s3 index తప్పు:** s3[i+j-1] కావాలి (0-based). s3[i+j] వాడితే off-by-one.
 - **మొదటి row/column base cases skip చెయ్యడం** – అవి "ఒక్క string మాత్రమే వాడిన" cases; వాటిని seed చెయ్యకపోతే inner recurrence కి foundation ఉండదు.
 - **Greedy గా match చెయ్యడం** ("s3 అక్షరం s1 లో ఉంటే s1 నుండి తీసుకో") – తప్పు, ఎందుకంటే రెండూ match అయ్యే చోట choice matters. తప్పకుండా రెండు branches (OR) explore.

11. Edit Distance (LeetCode #72) – Hard

- **సమస్య:** రెండు strings word1 , word2 ఇచ్చారు. word1 ని word2 గా మార్చడానికి కావాల్సిన **కనిష్ట operations** ఎంత? అనుమతించే operations: (1) **Insert** ఒక అక్షరం, (2) **Delete** ఒక అక్షరం, (3) **Replace** ఒక అక్షరం. (దీన్ని Levenshtein distance అంటారు.)
- **Key constraints:** $0 \leq word1.length, word2.length \leq 500$.
- **ఉదాహరణ:**

```

Input: word1 = "horse", word2 = "ros"
Output: 3

```

వివరణ: horse → rorse (h→r **replace**) → rose (r **delete**) → ros (e **delete**) = 3 ops.

- **ఎలా ఆలోచించాలి (5-step recipe):**

రెండు strings align చేస్తున్నాం → **2D state**. కీలక ప్రశ్న: word1 యొక్క మొదటి **i** అక్షరాలని, word2 యొక్క మొదటి **j** అక్షరాలుగా మార్చడానికి కనిష్ట ops ఎంత? చివరి అక్షరాలు (word1[i-1], word2[j-1]) చూడు –

- **సమానం అయితే:** ఏ operation అవసరం లేదు, dp[i-1][j-1] కి సమానం (చివరి అక్షరాలు free గా match).
- **వేరు అయితే:** మూడు choices లో కనిష్టం + 1 –
 - **Delete** word1[i-1] : dp[i-1][j] (word1 ని ఒక అక్షరం తగ్గించి align).
 - **Insert** word2[j-1] : dp[i][j-1] (word1 లో ఒక అక్షరం చేర్చినట్టు).
 - **Replace** word1[i-1] → word2[j-1] : dp[i-1][j-1] (రెండూ ఒక అక్షరం తగ్గి, ఇది replace).

1. **State:** dp[i][j] = word1[0..i-1] ని word2[0..j-1] గా మార్చే కనిష్ట ops.

2. **Recurrence:** word1[i-1] == word2[j-1] → dp[i][j] = dp[i-1][j-1]; లేకపోతే dp[i][j] = 1 + min(dp[i-1][j] /*delete*/, dp[i][j-1] /*insert*/, dp[i-1][j-1] /*replace*/).

3. **Base case:** dp[i][0] = i (word1 ని ఖాళీ చేయడానికి i deletes), dp[0][j] = j (ఖాళీ నుండి word2 build: j inserts).

4. **Order:** top-to-bottom, left-to-right.

5. **Answer:** dp[m][n] .

- **Brute Force (recursion):** ప్రతి position లో 3 branches → **O(3^{max(m,n)})**. State (i, j) పదే పదే overlapping.

- **Optimal Approach:**

- **Memoization:** edit(i, j) + 2D memo → O(m·n).
- **Tabulation:** (m+1)×(n+1) dp → **O(m·n)** time & space. 1D row (previous row ఒక్కటే అవసరం) తో O(n) space.

- **Solution (JavaScript):**

```
/**
 * @param {string} word1
 * @param {string} word2
 * @return {number} - కనిష్ట edit operations (Levenshtein)
 */
* dp[i][j] = word1[0..i) → word2[0..j) కనిష్ట ops.
* match → dp[i-1][j-1]; లేకపోతే 1 + min(delete, insert, replace).
*/
var minDistance = function (word1, word2) {
  const m = word1.length, n = word2.length;
  const dp = Array.from({ length: m + 1 }, () => new Array(n + 1).fill(0));

  // Base: ఒక string ఖాళీ అయితే, రెండోదాని పొడవు = ops (అన్నీ insert/delete)
  for (let i = 0; i <= m; i++) dp[i][0] = i; // i deletes
  for (let j = 0; j <= n; j++) dp[0][j] = j; // j inserts

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
```



```

    if (word1[i - 1] === word2[j - 1]) {
        dp[i][j] = dp[i - 1][j - 1]; // అక్షరాలు match → free
    } else {
        dp[i][j] = 1 + Math.min(
            dp[i - 1][j],      // delete word1[i-1]
            dp[i][j - 1],      // insert word2[j-1]
            dp[i - 1][j - 1]   // replace word1[i-1] → word2[j-1]
        );
    }
}
}

return dp[m][n];
};

```

- **Dry Run:** word1="horse" (m=5), word2="ros" (n=3). dp table (rows=h,o,r,s,e; cols="",r,o,s):

	""	r	o	s
""	0	1	2	3
h	1	1	2	3
o	2	2	1	2
r	3	2	2	2
s	4	3	3	2
e	5	4	4	3

Return dp[5][3] = 3 . ✓

- **Complexity:** Time $O(m \cdot n)$ – ప్రతి cell $O(1)$ work. Space $O(m \cdot n)$ (2D) లేదా $O(n)$ (1D rolling row).
- **గుర్తుంచుకోవాల్సినది:** Match → $dp[i-1][j-1]$; mismatch → $1 + \min(\text{delete}=dp[i-1][j], \text{insert}=dp[i][j-1], \text{replace}=dp[i-1][j-1])$. ఇది **string-alignment DP యొక్క king** – LCS, Distinct Subsequences, Wildcard/Regex Matching అన్నీ ఈ మూడు-neighbour template యొక్క variations. **మూడు neighbours = delete/insert/replace** అనే mapping ని గుర్తుంచుకుంటే recurrence మర్చిపోవు.
- **సాధారణ తప్పులు:**
 - **Base cases** ($dp[i][0]=i$, $dp[0][j]=j$) **skip చెయ్యడం** – 0 తో fill చేస్తే తప్పు (ఖాళీ నుండి build కి ops కావాలి). ఇదే అత్యంత common bug.
 - **మూడు neighbours లో ఒకటి miss చెయ్యడం** (సాధారణంగా diagonal $dp[i-1][j-1]$ replace ని) → over-count.
 - **s[i-1] vs s[i] offset:** dp index $i \leftrightarrow$ character index $i-1$. దీన్ని తారుమారు చేస్తే off-by-one.
 - Match అయినప్పుడు **+1** కలపడం – match free (0 cost), **+1** వద్దు.

12. Maximal Square (LeetCode #221) – Medium

- సమస్య: ఒక $m \times n$ binary matrix ('0' / '1' characters) ఇచ్చారు. అందులో అన్ని '1' లతో నిండిన అతిపెద్ద square యొక్క area (వైశాల్యం) ని కనుక్కో.
- Key constraints:** $1 \leq m, n \leq 300$. Matrix cells strings '0' / '1'.
- ఉదాహరణ:

Input: `[["1","0","1","0","0"],
["1","0","1","1","1"],
["1","1","1","1","1"],
["1","0","0","1","0"]]`

Output: 4

వివరణ: 2x2 square of 1's (side 2, area 2x2 = 4).

- ఎలా ఆలోచించాలి (5-step recipe):

ఇది subtle కానీ అందమైన DP. Square ని ఎలా track చెయ్యాలి? **Trick:** square యొక్క bottom-right మూల మీద state ని fix చెయ్యి. కీలక ప్రశ్న: cell (i, j) ని bottom-right corner గా కలిగిన అతిపెద్ద all-1's square యొక్క side ఎంత?

Cell (i,j) '1' అయితే, అక్కడ ఒక square ముగియాలంటే – దాని పైన (i-1,j), ఎడమ (i,j-1), diagonal (i-1,j-1) మూడు cells కూడా squares ముగించి ఉండాలి. ఆ మూడు squares లో అతి చిన్నది ఈ కొత్త square ని పరిమితం చేస్తుంది (bottleneck) – దానికి +1 (ఈ cell). ఇదే magic recurrence.

- State:** `dp[i][j]` = cell (i, j) ని bottom-right corner గా కలిగిన అతిపెద్ద all-1's square యొక్క side length.
 - Recurrence:** `matrix[i][j] == '1'` అయితే `dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])`; లేకపోతే 0.
 - Base case:** మొదటి row/column లో '1' అయితే `dp = 1` (వాటికన్నా పెద్ద square సాధ్యం కాదు). (Padding తో ఇది automatic.)
 - Order:** top-to-bottom, left-to-right.
 - Answer:** `maxSide2` – table లోని గరిష్ట side యొక్క square area.
- Brute Force:** ప్రతి cell ని corner గా తీసుకుని, సాధ్యమైన అన్ని square sizes ని expand చేసి check → $O((mn) \cdot \min(m,n)^2)$ లేదా అంతకంటే ఎక్కువ. చాలా slow.
 - Optimal Approach:**
 - Tabulation:** `dp[i][j]` = మూడు neighbours min + 1 → $O(m \cdot n)$ time. Padding ((m+1) × (n+1) , 0-filled) వాడితే boundary checks అవసరం లేదు – మొదటి row/col automatic గా 0. Space $O(m \cdot n)$, 1D తో $O(n)$.
 - (Memoization కూడా చెయ్యవచ్చు కానీ ఇది సహజంగా bottom-up.)
 - Solution (JavaScript):**

```
/**  
 * @param {character[][]} matrix - '0'/'1' strings  
 * @return {number} - అతిపెద్ద all-1's square యొక్క area
```

```

*
* dp[i][j] = (i,j) bottom-right corner గా ఉన్న square side.
* dp[i][j] = 1 + min(పైన, ఎడమ, diagonal). (1-indexed padding.)
*/
var maximalSquare = function (matrix) {
  const m = matrix.length, n = matrix[0].length;
  // (m+1)×(n+1) padding: row 0, col 0 అన్నీ 0 → boundary checks అక్కర్లేదు
  const dp = Array.from({ length: m + 1 }, () => new Array(n + 1).fill(0));
  let maxSide = 0;

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (matrix[i - 1][j - 1] === '1') { // matrix 0-indexed, dp 1-indexed
        // మూడు neighbours లో చిన్నది + ఈ cell
        dp[i][j] = 1 + Math.min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]);
        if (dp[i][j] > maxSide) maxSide = dp[i][j];
      }
      // '0' అయితే dp[i][j] 0 గానే ఉంటుంది (fill చేసినది)
    }
  }

  return maxSide * maxSide; // side² = area
};

```

- **Dry Run:** ఇచ్చిన matrix కి dp (padded; row0/col0 = 0 దాచాం):

dp	c1	c2	c3	c4	c5
r1 (1 0 1 0 0)	1	0	1	0	0
r2 (1 0 1 1 1)	1	0	1	1	1
r3 (1 1 1 1 1)	1	1	1	2	2
r4 (1 0 0 1 0)	1	0	0	1	0

గరిష్ట side = 2 (dp[3][4] లేదా dp[3][5]). Area = $2^2 = 4$. ✅

- **Complexity:** Time $O(m \cdot n)$ — ప్రతి cell $O(1)$. Space $O(m \cdot n)$ (2D) లేదా $O(n)$ (1D rolling row, diagonal ని ఒక temp లో పట్టుకుంటూ).
- **గుర్తుంచుకోవాల్సినది:** $dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$ (bottom-right corner state). "అతిపెద్ద square" = మూడు neighbours యొక్క minimum bottleneck + 1. Answer $side^2$. Maximal Rectangle (#85) వేరే (histogram + stack), కానీ square కి ఈ min-of-three అనేది elegant unique insight — దీన్ని మర్చిపోకు.
- **సాధారణ తప్పులు:**
 - **min బదులు max** వాడటం — square అన్ని దిక్కులా పూర్తిగా నిండాల్సి, కాబట్టి weakest (min) neighbour పరిమితి. Max వాడితే తప్పు.
 - **మూడో neighbour (diagonal $dp[i-1][j-1]$) ని మర్చిపోవడం** — కేవలం పైన & ఎడమ చూస్తే invalid squares లెక్కిస్తాం.

- **'1' (string) vs 1 (number) comparison** – LeetCode input strings; `matrix[i][j] === '1'` వాడాలి, `=== 1` కాదు.
- **Area vs side** గందరగోళం – return `side * side`, కేవలం `side` కాదు.

13. Best Time to Buy and Sell Stock III (LeetCode #123) – Hard

- **సమస్య:** `prices[i]` = i-వ రోజు stock ధర. **గరిష్టంగా రెండు transactions** (రెండు buy + రెండు sell) చేసి పొందగల **గరిష్ట profit** ఎంత? నియమం: ఒకేసారి **ఒక్క stock మాత్రమే** పట్టుకోగలము (కొత్తది కొనేముందు ఉన్నది అమ్మాలి).
- **Key constraints:** `1 <= prices.length <= 10^5`. Profit రాకపోతే `0`.
- **ఉదాహరణ:**

Input: `prices = [3,3,5,0,0,3,1,4]`
Output: `6`
 వివరణ: `day4(0)→day6(3) = 3`, తర్వాత `day7(1)→day8(4) = 3`. మొత్తం `6`.

- **ఎలా ఆలోచించాలి (5-step recipe) – State Machine DP:**

ఇది "at most K transactions" family ($K=2$). చాలా అందమైన approach: **state machine**. ప్రతి రోజు మనం 4 states లో ఏదో ఒకదానిలో ఉంటాం, ప్రతి state లో "ఇప్పటిదాకా best cash/profit" ని track చేస్తాం:

- **buy1** = మొదటి stock కొన్న తర్వాత cash (negative, ఎందుకంటే డబ్బు ఖర్చు) → maximize `-price`.
- **sell1** = మొదటి stock అమ్మిన తర్వాత profit → maximize `buy1 + price`.
- **buy2** = రెండో stock కొన్న తర్వాత (మొదటి profit వాడి) → maximize `sell1 - price`.
- **sell2** = రెండో stock అమ్మిన తర్వాత final profit → maximize `buy2 + price`.

ప్రతి రోజు ఈ 4 states ని update చేస్తూ వెళ్తాం. చివరికి `sell2` = answer.

1. **State:** `buy1, sell1, buy2, sell2` = ఇప్పటి రోజు దాకా ఆయా stage లో గరిష్ట cash/profit.
2. **Recurrence (ప్రతి price కి, ఈ order లో):** `buy1 = max(buy1, -price)`; `sell1 = max(sell1, buy1 + price)`; `buy2 = max(buy2, sell1 - price)`; `sell2 = max(sell2, buy2 + price)`.
3. **Base case:** `buy1 = buy2 = -Infinity` (ఇంకా కొనలేదు), `sell1 = sell2 = 0` (ఏ transaction లేకపోతే profit 0).
4. **Order:** ఎడమ నుండి కుడికి (ప్రతి రోజు); ప్రతి రోజు 4 updates ఇదే క్రమంలో.
5. **Answer:** `sell2`.

- **Brute Force:** అన్ని (`buy1, sell1, buy2, sell2`) day combinations → $O(n^4)$. Overlapping – "ఇక్కడినుండి 1 transaction max profit" పదే పదే.

- **Optimal Approach:**

- **General DP:** `dp[t][i]` = at most `t` transactions, day `i` దాకా max profit → $O(2 \cdot n) = O(n)$ time, $O(n)$ space (Problem 14 లో ఈ general form వస్తుంది).

- **State machine (4 variables):** పై insight → **$O(n)$ time, $O(1)$ space.** ఇదే అత్యంత elegant — కింది solution.

- **Solution (JavaScript):**

```
/**
 * @param {number[]} prices
 * @return {number} - గరిష్టంగా 2 transactions తో max profit
 *
 * 4-state machine: buy1 → sell1 → buy2 → sell2.
 * ప్రతి state = ఇప్పటిదాకా ఆ stage లో best cash/profit.
 */
var maxProfit = function (prices) {
  let buy1 = -Infinity; // మొదటి కొనుగోలు తర్వాత cash (max -price)
  let sell1 = 0;        // మొదటి అమ్మకం తర్వాత profit
  let buy2 = -Infinity; // రెండో కొనుగోలు తర్వాత (మొదటి profit వాడి)
  let sell2 = 0;        // రెండో అమ్మకం తర్వాత final profit

  for (const price of prices) {
    buy1 = Math.max(buy1, -price); // ఇప్పుడు కొంటే మంచిదా?
    sell1 = Math.max(sell1, buy1 + price); // మొదటిది ఇప్పుడు అమ్మితే?
    buy2 = Math.max(buy2, sell1 - price); // మొదటి profit తో రెండోది కొంటే?
    sell2 = Math.max(sell2, buy2 + price); // రెండోది ఇప్పుడు అమ్మితే?
  }

  return sell2;
};
```

ఒకే iteration లో ఈ order ఎందుకు OK? buy2 update కి sell1 (ఈ iteration లోనే update అయిన) ని వాడతాం — ఇది "అదే రోజు sell1 చేసి, అదే రోజు buy2" ని అనుమతిస్తుంది. అది profit లో ఏమీ కల్పించదు (net 0), కాబట్టి answer correct గానే ఉంటుంది. ఇదే classic accepted pattern.

- **Dry Run:** prices = [3, 3, 5, 0, 0, 3, 1, 4] . కీలక progression (buy1/sell1/buy2/sell2):

price	buy1	sell1	buy2	sell2
3	-3	0	-3	0
5	-3	2	-3	2
0	0	2	2	2
3	0	3	2	5
1	0	3	2	5
4	0	4	3	6

Return sell2 = 6. ✓

- **Complexity:** Time $O(n)$ — ఒక్క pass, ప్రతి రోజు 4 $O(1)$ updates. Space $O(1)$ — 4 variables.

- **గుర్తుంచుకోవాల్సినది: State machine: buy1→sell1→buy2→sell2 , ప్రతిది running max.** "At most K transactions" లో K చిన్నదైతే (K=2) ఈ hard-coded states; **K general అయితే (Problem 14) ఇదే idea ని loop లో generalize** చేస్తాం. Buy states negative cash, sell states profit – ఈ mental model గుర్తుంచుకో.
- **సాధారణ తప్పులు:**
 - **buy1 , buy2 ని 0 తో init చెయ్యడం** – **-Infinity** కావాలి (ఇంకా కొనని state ని invalid గా చూపాలి). 0 అయితే "0 ధరకి కొన్నట్టు" తప్పు profit.
 - **Update order తారుమారు** (sell2 ముందు, buy2 తర్వాత) – dependency chain (buy1→sell1→buy2→sell2) maintain చెయ్యాలి.
 - **prices reverse iterate** చేస్తే logic తలకిందులు. ఎడమ నుండి కుడికి (కాలక్రమంలో) మాత్రమే.

14. Best Time to Buy and Sell Stock IV (LeetCode #188) – Hard

- **సమస్య:** ఒక integer k మరియు prices[i] (i-వ రోజు ధర) ఇచ్చారు. **గరిష్టంగా k transactions** చేసి పొందగల **గరిష్ట profit** ఎంత? (ఒకసారి ఒక్క stock మాత్రమే.)
- **Key constraints:** $1 \leq k \leq 100$, $1 \leq \text{prices.length} \leq 1000$.
- **ఉదాహరణ:**

Input: k = 2, prices = [3,2,6,5,0,3]
Output: 7
 వివరణ: day2(2)→day3(6) = 4, day5(0)→day6(3) = 3. మొత్తం 7.

- **ఎలా ఆలోచించాలి (5-step recipe) – Generalized State Machine:**

Problem 13 (K=2) యొక్క నేరుగా generalization. అక్కడ 4 hard-coded variables ఉంటే, ఇక్కడ **k transactions** కి buy[1..k] , sell[1..k] arrays. ప్రతి రోజు, ప్రతి transaction level t కి:

- $\text{buy}[t] = t\text{-వ stock కొన్న తర్వాత best cash} = \max(\text{buy}[t], \text{sell}[t-1] - \text{price})$ (అంతకుముందు t-1 transactions profit వాడి కొంటాం).
- $\text{sell}[t] = t\text{-వ stock అమ్మిన తర్వాత best profit} = \max(\text{sell}[t], \text{buy}[t] + \text{price})$.

ఒక కీలక optimization: **k >= n/2 అయితే** – n రోజుల్లో గరిష్టంగా n/2 transactions మాత్రమే సాధ్యం. కాబట్టి k పెద్దదైతే అది "unlimited transactions" (Stock II) కి సమానం → ప్రతి upslope (prices[i] > prices[i-1]) ని greedy గా కూడు.

1. **State:** buy[t] = t-వ కొనుగోలు తర్వాత best cash; sell[t] = t-వ అమ్మకం తర్వాత best profit (ఇప్పటి రోజు దాకా).
2. **Recurrence (ప్రతి price, ప్రతి t=1..k):** $\text{buy}[t] = \max(\text{buy}[t], \text{sell}[t-1] - \text{price})$; $\text{sell}[t] = \max(\text{sell}[t], \text{buy}[t] + \text{price})$.
3. **Base case:** buy[t] = -Infinity (ఇంకా కొనలేదు), sell[0] = sell[t] = 0 (0 transactions → 0 profit). sell[0] ఎప్పుడూ 0 (anchor).

4. **Order:** ప్రతి price కి $t = 1$ నుండి k వరకు.

5. **Answer:** `sell[k]` .

- **Brute Force:** అన్ని k -transaction day combinations → **exponential**. State (t , day, holding) overlapping.

- **Optimal Approach:**

- **General DP:** `dp[t][holding]` rolling → **$O(n \cdot k)$** time, $O(k)$ space (కీందీ solution).

- **$k \geq n/2$ shortcut:** unlimited case ని $O(n)$ greedy తో – పెద్ద k కి array size (k) అనవసరంగా పెరగకుండా కాపాడుతుంది.

- **Solution (JavaScript):**

```
/**
 * @param {number} k - గరిష్ట transactions
 * @param {number[]} prices
 * @return {number} - max profit
 *
 * Problem 13 (k=2) యొక్క generalization: buy[t], sell[t] arrays.
 * k >= n/2 అయితే unlimited → greedy upslope sum.
 */
var maxProfit = function (k, prices) {
  const n = prices.length;
  if (n === 0 || k === 0) return 0;

  // k పెద్దదైతే: unlimited transactions (ప్రతి పెరుగుదల ని కూడు)
  if (k >= Math.floor(n / 2)) {
    let profit = 0;
    for (let i = 1; i < n; i++) {
      if (prices[i] > prices[i - 1]) profit += prices[i] - prices[i - 1];
    }
    return profit;
  }

  // buy[t] = t-వ కొనుగోలు తర్వాత best cash; sell[t] = t-వ అమ్మకం తర్వాత profit
  const buy = new Array(k + 1).fill(-Infinity);
  const sell = new Array(k + 1).fill(0); // sell[0] = 0 anchor

  for (const price of prices) {
    for (let t = 1; t <= k; t++) {
      buy[t] = Math.max(buy[t], sell[t - 1] - price); // t-1 profit తో కొను
      sell[t] = Math.max(sell[t], buy[t] + price); // t-వది అమ్ము
    }
  }

  return sell[k];
};
```

- **Dry Run:** $k = 2$, `prices = [3, 2, 6, 5, 0, 3]` . $n=6, k=2 < 3 \rightarrow$ DP branch. (`buy[1], sell[1], buy[2], sell[2]`):

price	buy[1]	sell[1]	buy[2]	sell[2]
3	-3	0	-3	0
2	-2	0	-2	0
6	-2	4	-2	4
5	-2	4	-1	4
0	0	4	4	4
3	0	4	4	7

Return `sell[2] = 7` . ✓ (k=2, [2,4,1] ఉదాహరణ → 2 కూడా ఇదే code correct.)

- **Complexity:** Time $O(n \cdot k)$ (shortcut branch $O(n)$). Space $O(k)$ — buy/sell arrays.
- **గుర్తుంచుకోవాల్సినది:** `buy[t] = max(buy[t], sell[t-1] - price)` , `sell[t] = max(sell[t], buy[t] + price)` , **answer** `sell[k]` . ఇది Problem 13 యొక్క loop-generalization — **stock DP కుటుంబం మొత్తం ఒకే state machine** (Stock I = k=1, II = k=∞, III = k=2, IV = k). $k \geq n/2 \rightarrow$ unlimited shortcut ని మర్చిపోకు (లేకపోతే పెద్ద k కి అనవసర work/memory).
- **సాధారణ తప్పులు:**
 - $k \geq n/2$ **shortcut skip చెయ్యడం** — k చాలా పెద్దదైతే $O(n \cdot k)$ array అనవసరంగా పెద్దది, కొన్ని edge inputs లో TLE/memory issue.
 - `sell[0]` **ని ప్రత్యేకంగా 0 గా ఉంచకపోవడం** — t=1 కి `sell[t-1] = sell[0]` కావాలి; ఇది "0 transactions, 0 profit" anchor. తప్పితే మొత్తం chain తప్పు.
 - **Inner loop ని t = k నుండి 1 కి తిప్పడం** — ఇక్కడ ascending (1 → k) పని చేస్తుంది (buy[t] కి same-day sell[t-1] allowed). Order మార్చే ముందు జాగ్రత్త.
 - **k = 0 edge** — 0 transactions → 0 profit; ముందే return.

ముగింపు — DP ని ఎలా గెలవాలి (Cheat-sheet)

అభినందనలు! ఇక్కడిదాకా వస్తే, నువ్వు DP యొక్క అన్ని ప్రధాన patterns చూశావు. చివరిగా ఒక్కసారి మనసులో ముద్రించుకో:

Pattern	State $dp[\dots]$	Recurrence గుండ్	ఉదాహరణ problems
Fibonacci-style 1D	$dp[i] = i$ దాకా answer	$dp[i] = f(dp[i-1], dp[i-2])$	Climbing Stairs, House Robber
Prefix boolean/segment	$dp[i] =$ మొదటి i valid?	$dp[i] = OR_j (dp[j] \& \text{piece})$	Word Break
Unbounded knapsack	$dp[a] =$ amount a కి min/count	$dp[a] = \min/sum_c (dp[a-c])$	Coin Change
Subsequence ending at i	$dp[i] = i$ వద్ద ముగిసే	$dp[i] = 1 + \max(dp[j])$, $ans = \max(dp)$	LIS
Grid path	$dp[i][j] = (i,j)$ దాకా	$dp[i][j] = \text{grid} + \min/sum(\text{పైన, ఎడమ})$	Min Path Sum, Unique Paths II, Triangle
Interval/palindrome	$dp[i][j] = s[i..j]$ palindrome	$s[i]==s[j] \& dp[i+1][j-1]$	Longest Palindromic Substring
Two-string align	$dp[i][j] = s1[0..i]$ vs $s2[0..j]$	match $\rightarrow$ diag; else $\min/or(3 \text{ neighbours})$	Edit Distance, Interleaving
Corner-anchored 2D	$dp[i][j] = (i,j)$ corner square	$1 + \min(3 \text{ neighbours})$	Maximal Square
State machine	buy[t]/sell[t] per stage	running max across states	Stock III, Stock IV

Interview లో DP వచ్చినప్పుడు ఈ 5 అడుగులు బిగ్గరగా చెప్పు:

1. "State ఏమిటి? $dp[\dots]$ అంటే సరిగ్గా ఇది..." (ఇక్కడ time తీసుకో — ఇదే 90% పని).
2. "Recurrence: ఈ state దగ్గర choices ఇవి..., వాటిలో max/min/sum/or..."
3. "Base case: అన్నిటికన్నా చిన్నది..."
4. "Fill order: ఇలా (చిన్న $\rightarrow$ పెద్ద), ఎందుకంటే dependencies..."
5. "Answer: $dp[\dots]$ వద్ద."

తర్వాత "brute-force recursion $\rightarrow$ memoize $\rightarrow$ tabulate $\rightarrow$ space optimize" అని evolution చూపిస్తే — interviewer కి నీ thinking process పూర్తిగా కనిపిస్తుంది. **DP అంటే బట్టి కాదు, ఈ framework.** ఇది ఒంటబట్టింది కాబట్టి, ఇక ఏ DP problem నీ ముందు నిలబడలేదు. All the best! 🚀